

EEE 475 - Medical Imaging Reconstruction and Processing

Homework 2

Tuna Şahin, 22201730

Contents

0	Introduction	2
0.1	GenAI Usage Disclosure	2
1	Question 1	2
1.1	Question 1.1	2
1.2	Question 1.2	3
1.3	Question 1.3	4
1.4	Question 1.4	5
1.5	Question 1.5	6
1.6	Question 1.6	7
1.7	Question 1.7	8
1.8	Question 1.8	9
1.9	Question 1.9	11
1.10	Question 1.10	11
1.11	Question 1.11	13
1.12	Question 1.12	14
2	Question 2	16
2.1	Question 2.1	16
2.2	Question 2.2	17
2.3	Question 2.3	18
2.4	Question 2.4	19
2.5	Question 2.5	21
2.6	Question 2.6	22
2.7	Question 2.7	23
3	Question 3	24
3.1	Question 3.1	24
3.2	Question 3.2	25
3.3	Question 3.3	25
3.4	Question 3.4	26
4	Conclusion	28
A	Appendix A: Images with Normalisation Correction	29
A.1	Question 1.8	29
A.2	Question 1.9	29
A.3	Question 2.3	30
A.4	Question 2.7	31
A.5	Comments on Normalisation Correcting	31
B	Appendix B: MATLAB Script and Functions	31

Introduction

In this assignment, we are presented with different data in the form of k-space trajectories or direct reference images. Specifically, we are given the spiral k-space acquisition [1] of a phantom for question 1, and for questions 2 and 3, we are working with the *Modified Shepp-Logan* Phantom. We are given the original phantom and a radial MRI acquisition [2] of the phantom where the trajectories start from the origin as opposed to starting from $-k_{extend}$. We then reconstruct the data with different methods and compare them with the reference image.

The error maps in this assignment were all plotted in the interval $[0, 0.2]$ unless specified otherwise.

For the programming aspects of this assignment, MATLAB was preferred. Repetitive sections of code, such as direct summation, interpolation, image comparison, and plotting overlapping cross sections, were implemented as functions to avoid duplication. These functions, along with the entire MATLAB code is available in Appendix B.

GenAI Usage Disclosure

No GenAI was used for the purpose of this homework for the purpose of algorithm development and problem-solving. GenAI was used solely to conduct a literature review and to brainstorm the plausibility of the outcomes.

Question 1

In this question, we are given a spiral k-space trajectory and its corresponding samples. We are then tasked with reconstructing this data using several reconstruction methods discussed in the lectures. These methods are Direct Summation, K-Space Convolution, and Interpolation. We then compare the PSNR and SSIM of these methods' results and their computation times.

Question 1.1

We are given a non-Cartesian spiral k-space trajectory. The data consists of 6 separate acquisitions, each with 2048 samples. The trajectories are denser for lower-frequency components to emphasise them, since most MRI images consist mostly of lower frequencies.

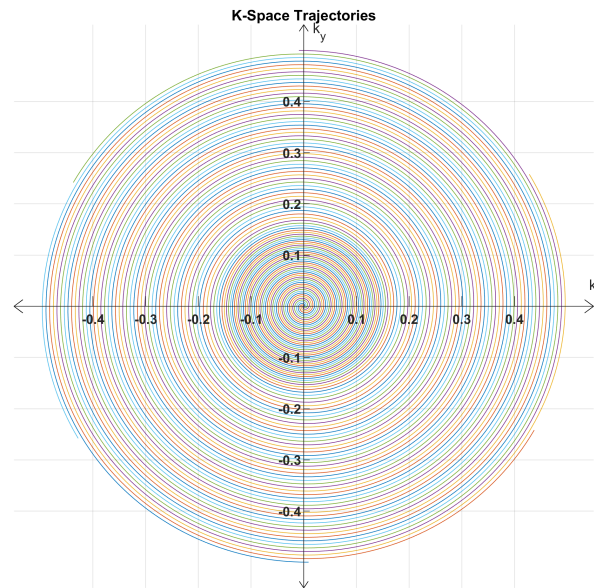


Figure 1.1.1: Spiral K-Space Trajectories

```

1 spiral_data = load("spiral.mat");
2 clf;
3 fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"K-Space
  ↳ Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[600 900],"", "-");
4 grid on
5
6 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.85]);
7 if save_image
8     exportgraphics(gcf, strcat("q11_trajectory.png"), 'Resolution', 300)
9 end

```

Question 1.2

By using the *voronoidens.m* function [3] provided along with the homework, we are able to obtain the area of each cell corresponding to each sample. However, with this method, 186 cells have infinite area. This is due to the fact that the samples at the edge of the acquired segment of k-space have infinite area, as their corresponding Voronoi regions extend to infinity due to the absence of neighboring samples beyond the boundary. This is visualised in Figure 1.2.1 where areas diverge to infinity.

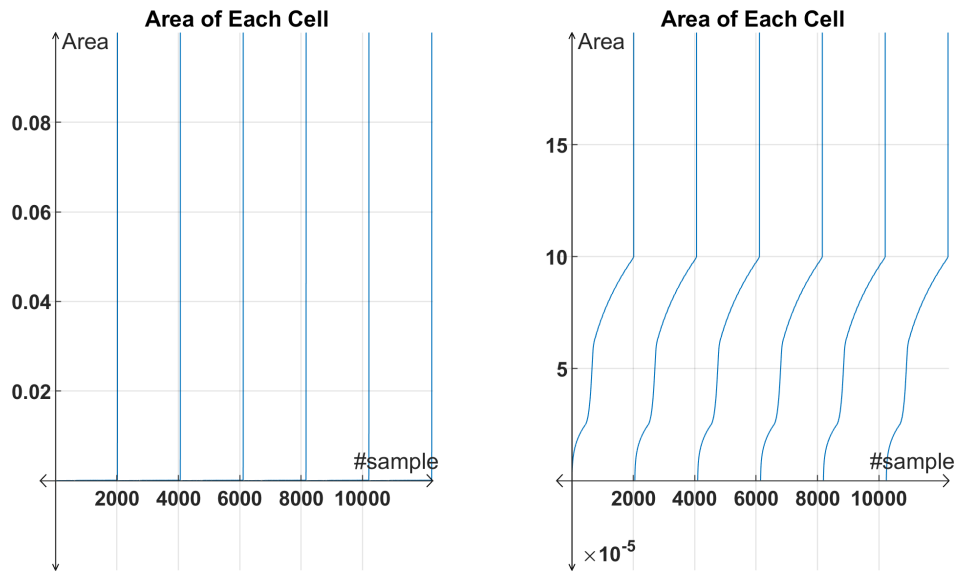


Figure 1.2.1: Area of Cells per Each Sample

```

1 area = voronoidens(spiral_data.ktraj);
2 total_nans = sum(isnan(area), 'all')
3 subplot(1,2,1)
4 fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample","Area",[-500
  ↳ length(area(:))],[-0.02 0.1],"off",[800 400],"", "-");
5 grid on
6 subplot(1,2,2)
7 fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample","Area",[-500
  ↳ length(area(:))],[-0.00004 0.0002],"off",[800 400],"", "-");
8 grid on
9 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
10 if save_image
11     exportgraphics(gcf, strcat("q12_area.png"), 'Resolution', 300)
12 end

```

Question 1.3

To address the issue of the samples' areas diverging to infinity, we can add a border to limit the area of all samples. We add a circle of radius 0.51 to surround the acquired k-space, which is a disc of radius 0.5. This method bounds the areas of samples for the original trajectory while the area for samples on the circle were infinite. We can discard the area values for the samples on the circle as they were only used to limit the areas of the original samples. The results obtained with this method are visible in Figure 1.3.2

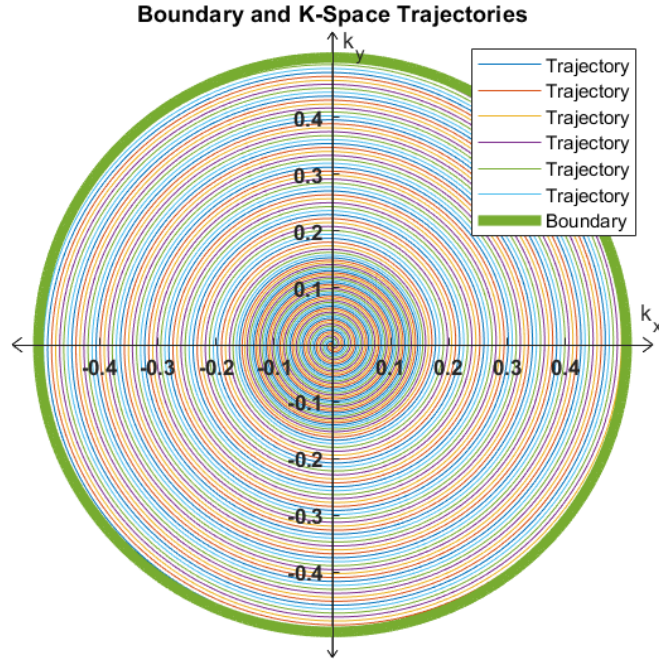


Figure 1.3.1: Bounded Spiral K-Space Trajectories

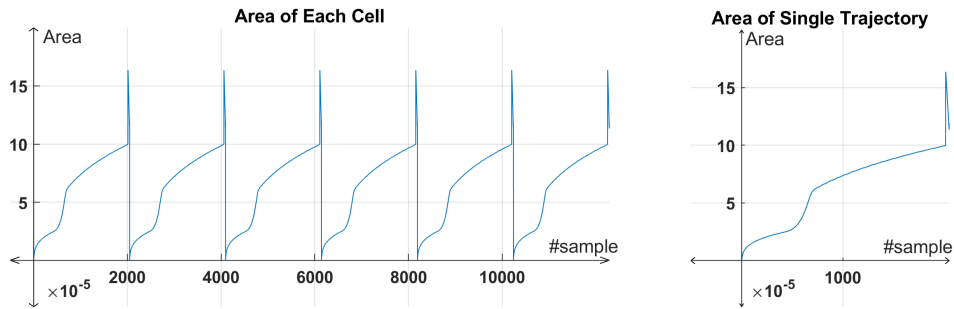


Figure 1.3.2: Bounded Area of Cells per Each Samples


```

1  clf;
2  circle = 0.505*exp(1*j*linspace(0,2*pi,2049));
3  circle = circle(1:end-1);
4  limited_ktraj=[spiral_data.ktraj circle.'];
5  fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"Boundary and K-Space
   ↳ Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"on",[400
   ↳ 800],"Trajectory","-");
6  plot(circle,"LineWidth",5,'displayName',"Boundary")
7  legend()
8  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.85]);
9  if save_image
10     exportgraphics(gcf, strcat("q13_trajectory.png"), 'Resolution', 300)
11 end
12 clf;
13 area2 = voronoidens(limited_ktraj);
14 area2 = area2(:,1:6);
15 subplot(1,3,1:2)
16 fineplot(1:length(area2(:)),area2(:),"Area of Each Cell", "#sample", "Area", [-500
   ↳ length(area2(:))], [-0.00004 0.0002], "off", [800 400], "", "-");
17 grid on
18 subplot(1,3,3)
19 fineplot(1:length(area2(:,1)),area2(:,1),"Area of Single
   ↳ Trajectory", "#sample", "Area", [-500 length(area2(:,1))], [-0.00004
   ↳ 0.0002], "off", [800 400], "", "-");
20 grid on
21 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
22 if save_image
23     exportgraphics(gcf, strcat("q13_area.png"), 'Resolution', 300)
24 end

```

Question 1.4

We compute the 128x128 Image by directly summing over all the k-space data. This is to be used as the reference image throughout question 1, as it is the most accurate reconstruction method but takes the most time. This method is therefore referred to as the gold standard, while we continue to search for faster yet more accurate reconstruction methods.

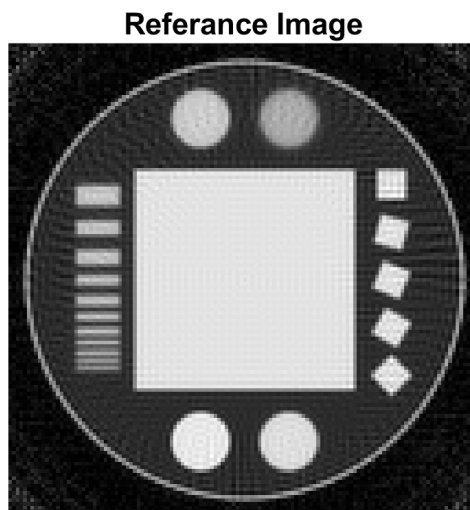


Figure 1.4.1: 128x128 Image Reconstructed by Direct Summation

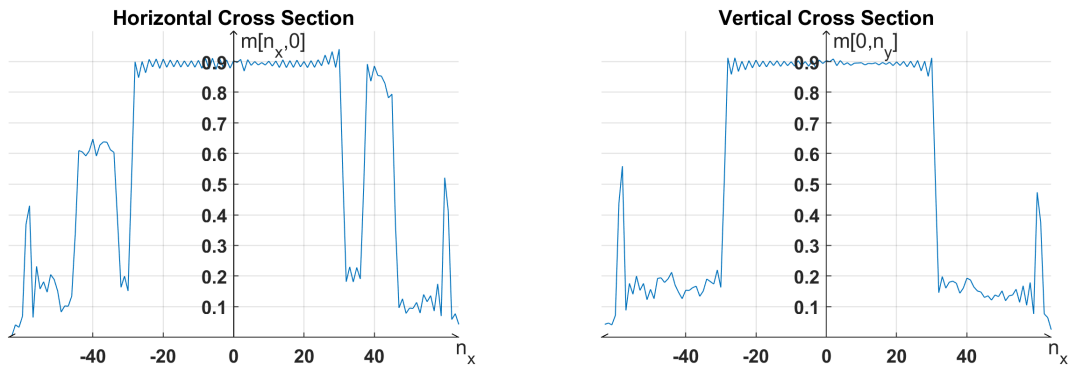


Figure 1.4.2: Vertical and Horizontal Cross-Sections of the Image

Processing Time = 2.91 seconds, CPU Time = 20.9212.

```

1  N=128;
2  t0 = cputime;
3  ima_direct = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,128,1);
4  time_elapsed1 = t0 - cputime
5  clf;
6  imshow(ima_direct)
7  title("Reference Image")
8  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
9  if save_image
10     exportgraphics(gcf, strcat("q14_image.png"), 'Resolution', 300)
11 end
12 clf;
13 subplot(1,2,1)
14 fineplot([1:N]-N/2,abs(ima_direct(end/2,:)), "Horizontal Cross
    ↳ Section", "n_x", "m[n_x,0]", [-N/2 N/2], [0 1], "off", [800 400], "", "-");
15 grid on
16 subplot(1,2,2)
17 fineplot([1:N]-N/2,abs(ima_direct(:,end/2)), "Vertical Cross
    ↳ Section", "n_x", "m[0,n_y]", [-N/2 N/2], [0 1], "off", [800 400], "", "-");
18 grid on
19 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
20 if save_image
21     exportgraphics(gcf, strcat("q14_cross.png"), 'Resolution', 300)
22 end

```

Question 1.5

In order to see the effect of density compensation, we compute the 128x128 Image by direct summation without density compensation. The result features an image with low-frequency emphasis, which results in a blurry, hazy appearance. The error for this question was plotted in the interval $[0, 1]$ in addition to the standard $[0, 0.2]$.

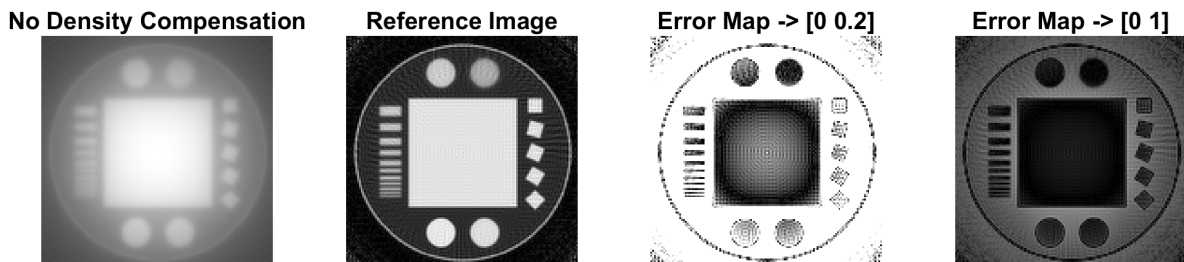


Figure 1.5.1: Comparison of No Density Compensated Image with the Reference Image

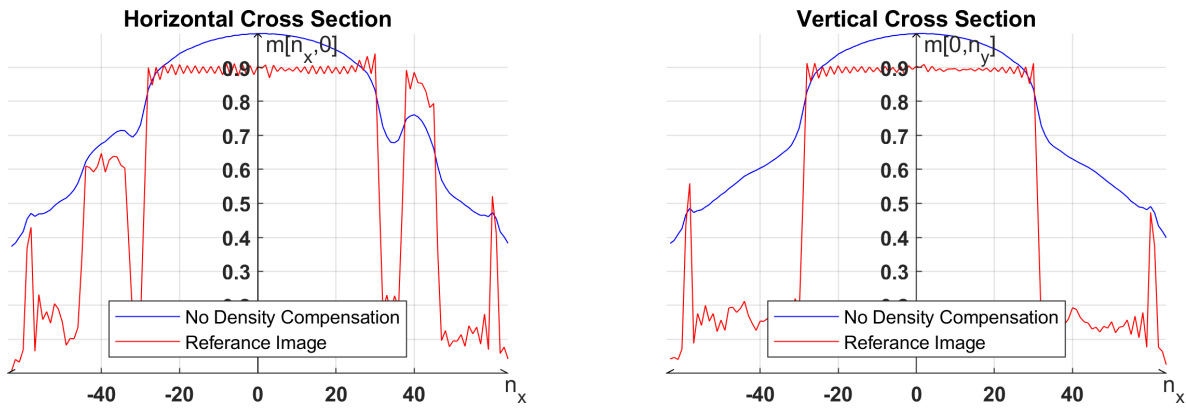


Figure 1.5.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 10.8638, SSIM = 0.2995, Processing Time = 2.7952 seconds, CPU Time = 23.

```

1 N=128;
2 t0 = cputime;
3 ima_direct2 =
4     ↳ direct_summation(spiral_data.ktraj,spiral_data.kdata,ones(size(spiral_data.kdata)),128,1);
5 time_elapsed2 = t0 - cputime
6 plot_comparative_images(ima_direct2,ima_direct,save_image,"q15_comp","No Density Compensation")
7 plot_cross_sections(ima_direct2,ima_direct,save_image,"q15_cross","No Density Compensation")
8 PSNR2 = psnr(abs(ima_direct2),ima_direct)
9 SSIM2 = ssim(abs(ima_direct2),ima_direct)

```

Question 1.6

We can also use the direct summation approach to calculate values for more points. We compute the 256x256 image with double the FOV and the same resolution. The resulting image is the extension of the reference image. This method calculates the central 128x128 image with the same values as the reference image; therefore, the central 128x128 image must be identical to the reference image. This results in an infinite PSNR and an SSIM of 1. This method provides the same information as the reference image and no additional information since the region covered by the additional FOV is mostly empty. Therefore, this increase in computation time is not worth it.

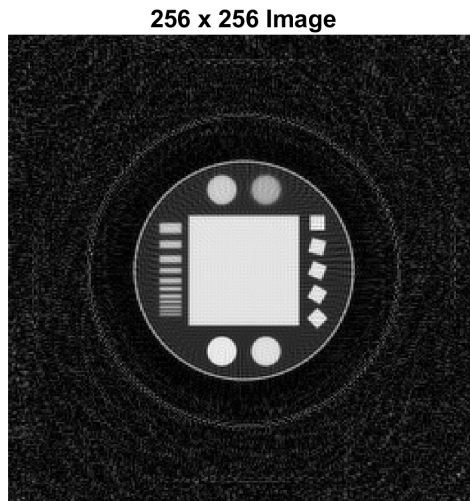


Figure 1.6.1: 256x256 Direct Summation Image

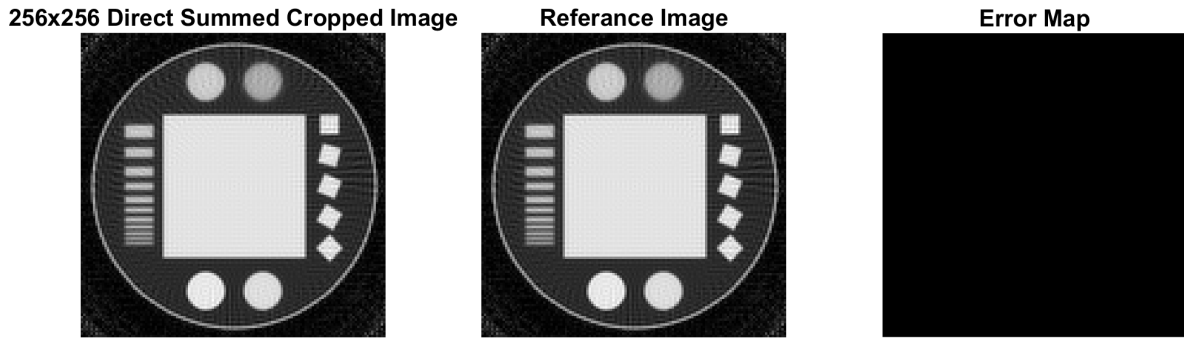


Figure 1.6.2: Comparison of Cropped Image with the Reference Image

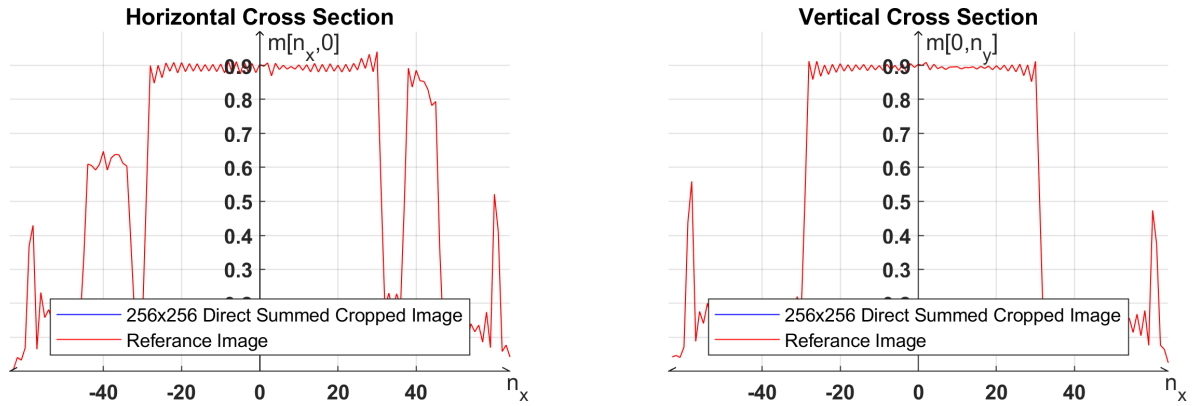


Figure 1.6.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = ∞ , SSIM = 1, Processing Time = 11.6163 seconds, CPU Time = 86.0156.

```

1  N3 = 256;
2  t0 = cputime;
3  ima_direct3 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,256,1);
4  time_elapsed3 = t0 - cputime
5  cropped = ima_direct3(N3/4+1:3*N3/4,N3/4+1:3*N3/4);
6  clf;
7  imshow(ima_direct3)
8  title("256 x 256 Image")
9  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
10 if save_image
11     exportgraphics(gcf, strcat("q16_image.png"), 'Resolution', 300)
12 end
13 plot_comparative_images(cropped,ima_direct,save_image,"q16_comp","256x256 Direct
    ↳ Summed Cropped Image")
14 plot_cross_sections(cropped,ima_direct,save_image,"q16_cross","256x256 Direct Summed
    ↳ Cropped Image")
15 PSNR3 = psnr(abs(cropped),ima_direct)
16 SSIM3 = ssim(abs(cropped),ima_direct)

```

Question 1.7

This method doubles the pixel density instead of doubling the FOV. This results in more pixels in the reconstructed image. However, this does not result in an increase in resolution as it is given by $\delta_{x,y} = 1/k_{extend}$ which is 1. So this increase in pixel density cannot be supported by our current k-space data. If our k-space data had higher frequency components, then this method would result in a higher resolution image but again with our current data we do not gain any additional information.

This is supported by the fact that $SSIM \approx 1$. This small difference is probably due to floating-point errors or similar error residues. Again, we did not gain any additional information in addition to the reference image so this increase in processing time was again not worth it.

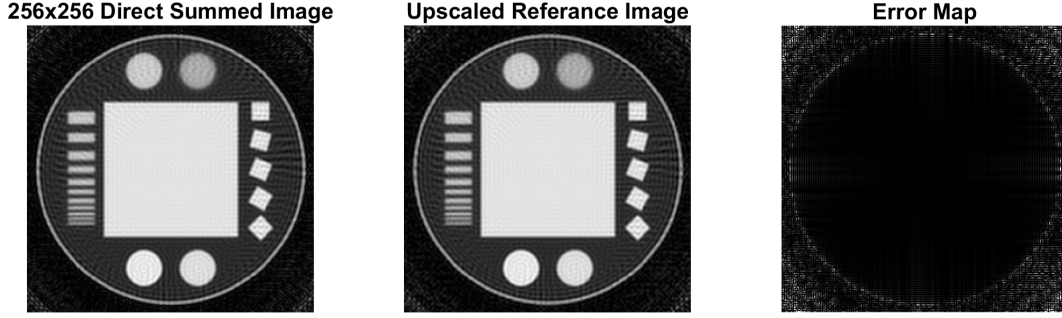


Figure 1.7.1: Comparison of 256x256 Image with the Upscaled Reference Image

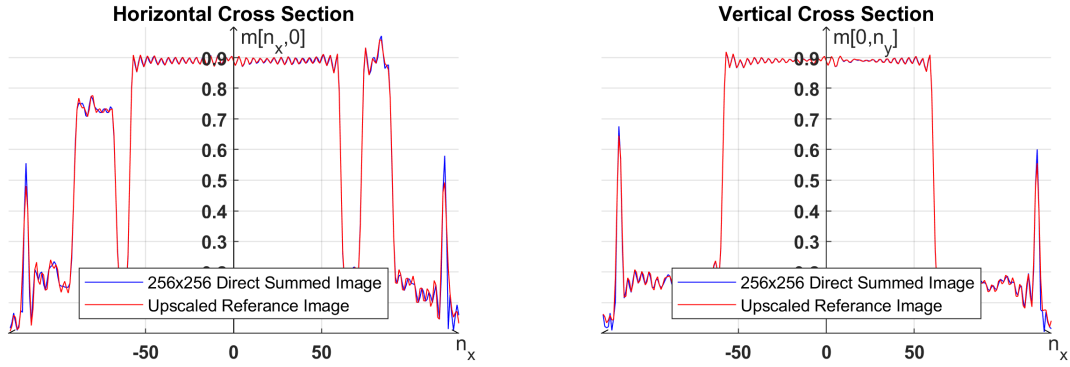


Figure 1.7.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 34.2773, SSIM = 0.9236, Processing Time = 11.6666 seconds, CPU Time = 84.25.

```

1  N4 = 256;
2  t0 = cputime;
3  ima_direct4 = direct_summation(spiral_data.ktraj, spiral_data.kdata, area2, 256, 0.5);
4  time_elapsed4 = t0 - cputime;
5  k_ref = fft2c(ima_direct);
6  k_ref_padded = zeros([N4 N4]);
7  k_ref_padded(N4/4+1:(3*N4/4), N4/4+1:(3*N4/4)) = k_ref;
8  ima_direct_us = ifft2c(k_ref_padded);
9  ima_direct_us = ima_direct_us / max(ima_direct_us(:));
10 plot_comparative_images(abs(ima_direct4), abs(ima_direct_us), save_image, "q17_comp", "256x256
    ↳ Direct Summed Image", "Upscaled Reference Image")
11 plot_cross_sections(ima_direct4, ima_direct_us, save_image, "q17_cross", "256x256 Direct Summed
    ↳ Image", "Upscaled Reference Image")
12 PSNR4 = psnr(abs(ima_direct4), abs(ima_direct_us))
13 SSIM4 = ssim(abs(ima_direct4), abs(ima_direct_us))

```

Question 1.8

We decrease the pixel density while keeping the FOV constant. This decreases the resolution of the image as we are not utilising the high-frequency components, as this decrease in pixel density only uses half of the k-space corresponding to the interval $[-0.25, 0.25]$. This significantly reduces the processing time while also sacrificing critical information. So overall, this in processing time reduction was not worth the information loss in my opinion. Also, we can notice that the cross sections do not overlap completely, possibly due to issues while normalising. We can scale the image with a constant to counter this issue. The results of this are available in Section A.1.

64x64 Image

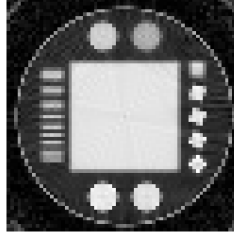


Figure 1.8.1: Double Pixel Size 64x64 Image

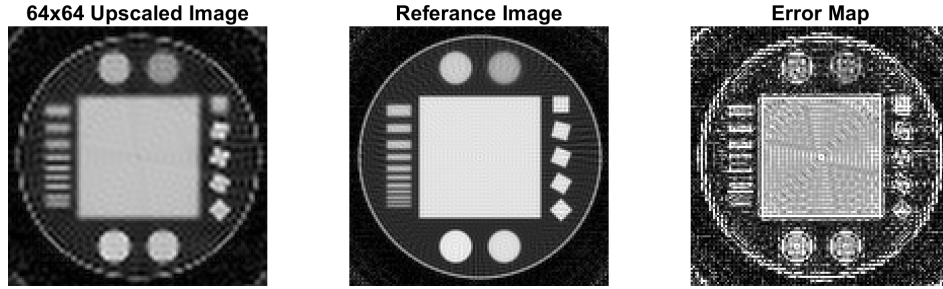


Figure 1.8.2: Comparison of Upscaled 64x64 Image with the Reference Image

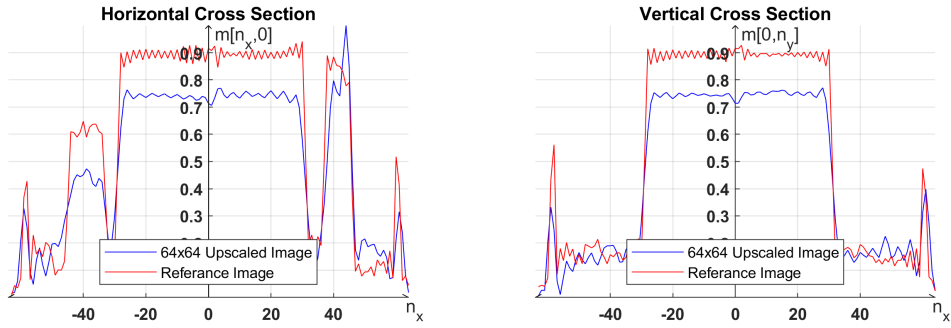


Figure 1.8.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 19.1969, SSIM = 0.6534, Processing Time = 0.8937 seconds, CPU Time = 5.6875.

```

1  t0 = cputime;
2  ima_direct5 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,64,2);
3  time_elapsed5 = t0 - cputime;
4  k_data_5 = fft2c(ima_direct5);
5  k_data_5_padded = zeros([N,N]);
6  k_data_5_padded(N/4+1:3*N/4,N/4+1:3*N/4) = k_data_5;
7  ima_direct5_us = ifft2c(k_data_5_padded);
8  ima_direct5_us = abs(ima_direct5_us)/max(abs(ima_direct5_us(:)));
9  % ima_direct5_us = ima_direct5_us * 1.2; %normalisation correction? comment to disable
10
11  clf;
12  imshow(ima_direct5)
13  title("64x64 Image")
14  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
15  if save_image
16      exportgraphics(gcf,strcat("q18_image.png"),'Resolution',300)
17  end
18  plot_comparative_images(ima_direct5_us,ima_direct,save_image,"q18_comp","64x64 Upscaled Image")
19  plot_cross_sections(ima_direct5_us,ima_direct,save_image,"q18_cross","64x64 Upscaled Image")
20  PSNR5 = psnr(abs(ima_direct5_us),ima_direct)
21  SSIM5 = ssim(abs(ima_direct5_us),ima_direct)

```


Question 1.9

In this question, rather than the direct summation approach, we will use a gridding approach with the provided MATLAB function [4]. We convolve the non-Cartesian k-space trajectory with a kernel and resample the resulting data onto a Cartesian grid. This results in significant reductions in processing times. In this approach, however, we sacrifice some image quality as well, mainly in the form of noise and SNR loss. Again, in this question, we can notice that the cross sections do not overlap properly. The compensated versions are available in Section A.2

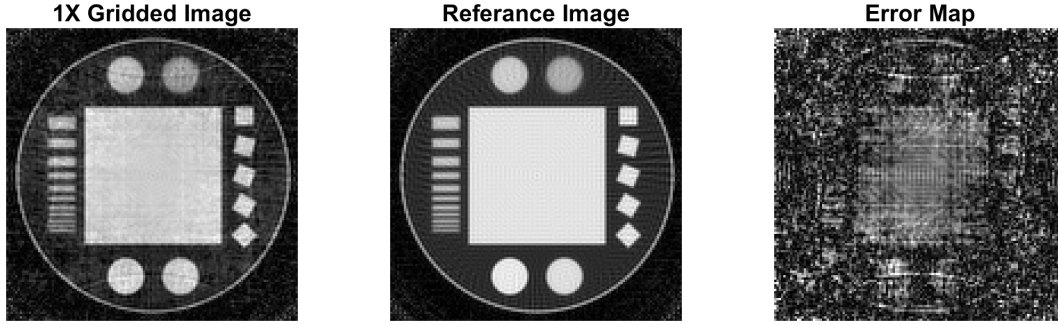


Figure 1.9.1: Comparison of 1X Gridded Image with the Reference Image

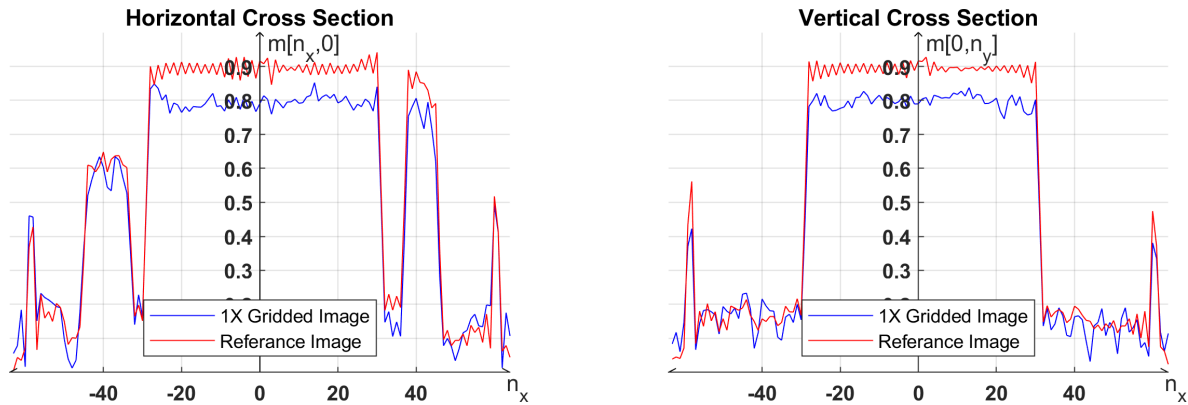


Figure 1.9.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 25.7217, SSIM = 0.7564, Processing Time = 0.0117 Seconds, , CPU Time = 10.3594.

```

1  t0 = cputime;
2  ima9 = gridkb(spiral_data.kdata,spiral_data.ktraj,area2,N,1,2,'image');
3  time_elapsed6 = t0 - cputime
4  ima9 = abs(ima9).'/max(abs(ima9(:)));
5  % ima9 = ima9 * 1.08; % for normalisation correcting. comment to disable
6  plot_comparative_images(ima9,ima_direct,save_image,"q19_comp","1X Gridded Image")
7  plot_cross_sections(ima9,ima_direct,save_image,"q19_cross","1X Gridded Image")
8  PSNR6 = psnr(abs(ima9),ima_direct)
9  SSIM6 = ssim(abs(ima9),ima_direct)

```

Question 1.10

We again implement a gridding method similar to question 1.9, but we use an oversampling factor of 2. This allows us to acquire higher accuracy images while increasing the processing times. This increase in processing times are negligible and the increase in performance is quite significant compared to the previous results.

2X Gridded Image

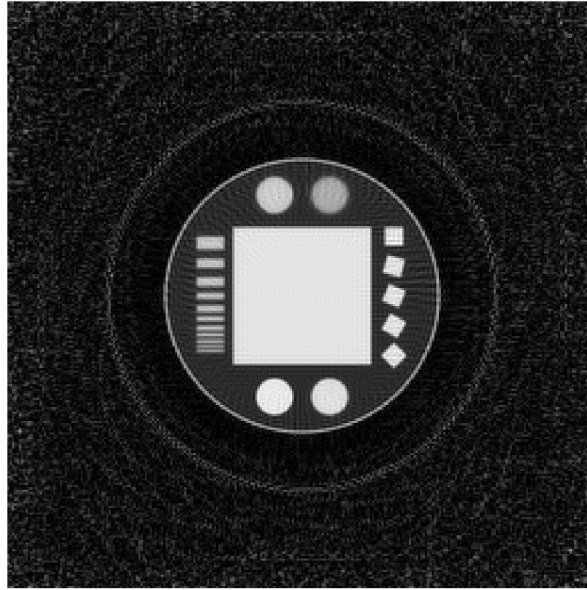
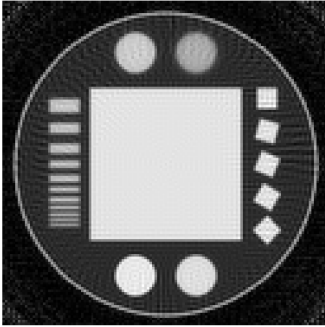
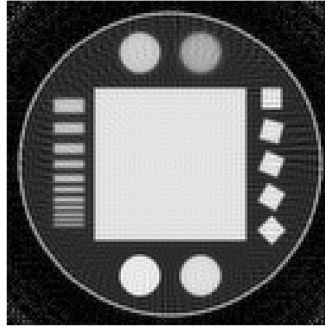


Figure 1.10.1: 2X Gridded Image

2X Gridded Cropped Image



Reference Image



Error Map



Figure 1.10.2: Comparison of 2X Gridded Cropped Image with the Reference Image

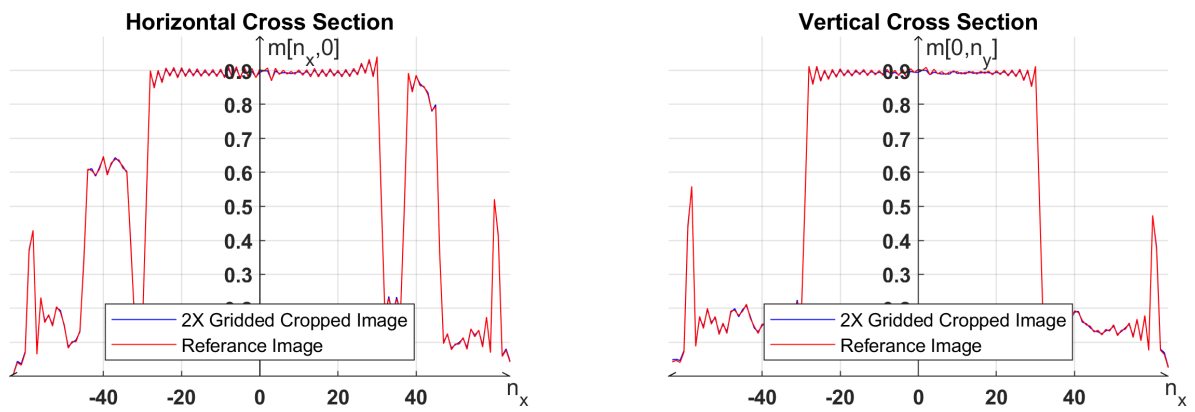


Figure 1.10.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 49.0465, SSIM = 0.9956, Processing Time = 0.0501 seconds, CPU Time = 15.2969.

```

1 t0 = cputime;
2 ima10 = gridkb(spiral_data.kdata,spiral_data.ktraj,area2,N,2,4,'image');
3 time_elapsed7 = t0 - cputime
4 ima10 = abs(ima10).'/max(abs(ima10(:)));
5 clf;
6 imshow(ima10)
7 title("2X Gridded Image")
8 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
9 if save_image
10     exportgraphics(gcf, strcat("q110_image.png"), 'Resolution', 300)
11 end
12 ima10 = ima10(65:192,65:192);
13 plot_comparetive_images(ima10,ima_direct,save_image,"q110_comp","2X Gridded Cropped
    ↪ Image")
14 plot_cross_sections(ima10,ima_direct,save_image,"q110_cross","2X Gridded Cropped
    ↪ Image")
15 PSNR7 = psnr(abs(ima10),ima_direct)
16 SSIM7 = ssim(abs(ima10),ima_direct)

```

Question 1.11

In this question, we again implement another reconstruction method, interpolation. We interpolate the non-Cartesian samples to the nearest points on the Cartesian grid. Compared to the previous methods, this algorithm provided worse SSIM and PSNR values while offering a small decrease in processing times.

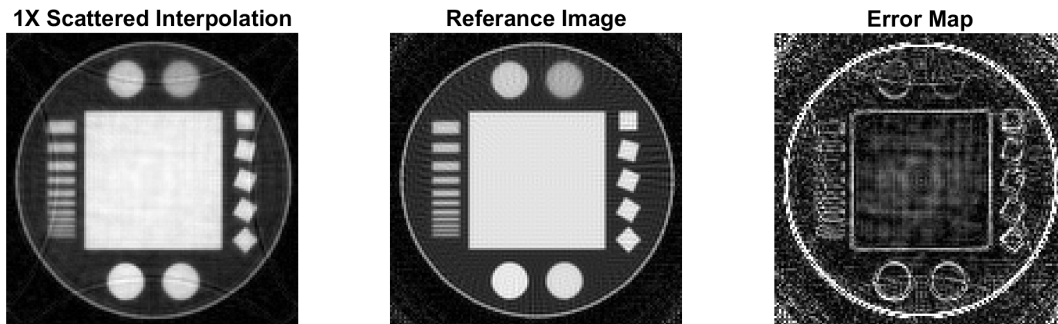


Figure 1.11.1: Comparison of 1X Interpolated Image with the Reference Image

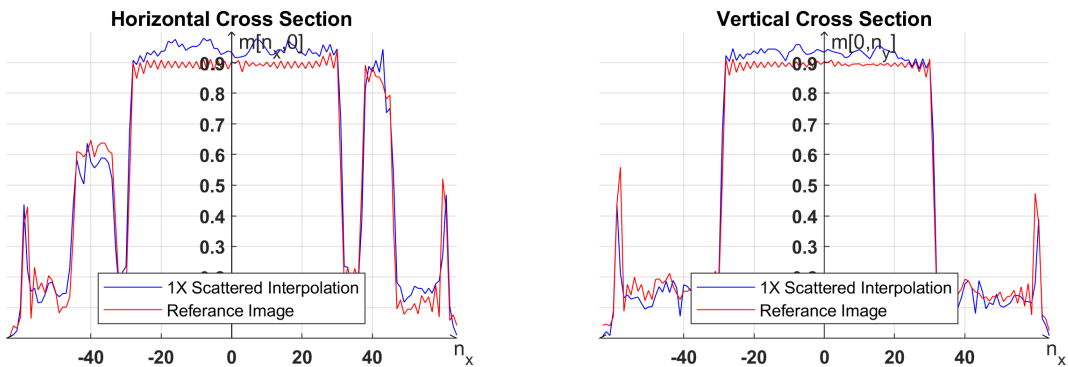


Figure 1.11.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 22.9973, SSIM = 0.6955, Processing Time = 0.0544 seconds, CPU Time = 19.3125.

```

1 t0 = cputime;
2 ima_interp = scattered_interpolate(spiral_data.ktraj,spiral_data.kdata,128);
3 time_elapsed8 = t0 - cputime
4 plot_compartative_images(ima_interp,ima_direct,save_image,"q111_comp",'1X Scattered
  ↳ Interpolation')
5 plot_cross_sections(ima_interp,ima_direct,save_image,"q111_cross",'1X Scattered
  ↳ Interpolation')
6 PSNR8 = psnr(abs(ima_interp), ima_direct)
7 SSIM8 = ssim(abs(ima_interp), ima_direct)

```

Question 1.12

This time, we utilize interpolation with an oversampling factor of 2. This method offers decent PSNR and SSIM performance and significantly short processing times.

2X Interpolated Image

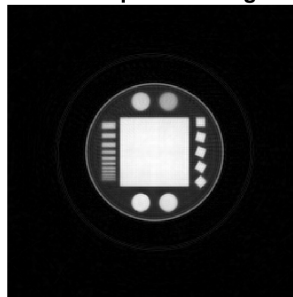
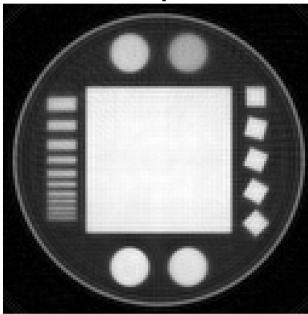
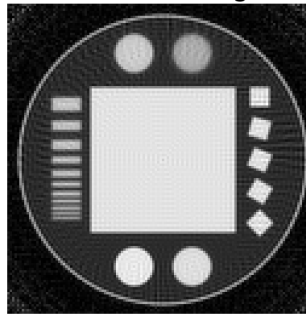


Figure 1.12.1: 2X Interpolated Image

2X Scattered Interpolation Cropped



Reference Image



Error Map

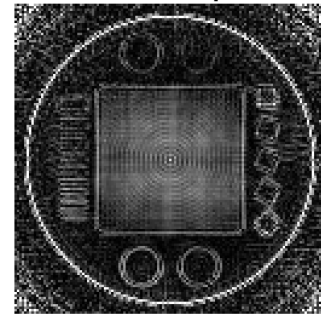


Figure 1.12.2: Comparison of 2X Interpolated Cropped Image with the Reference Image

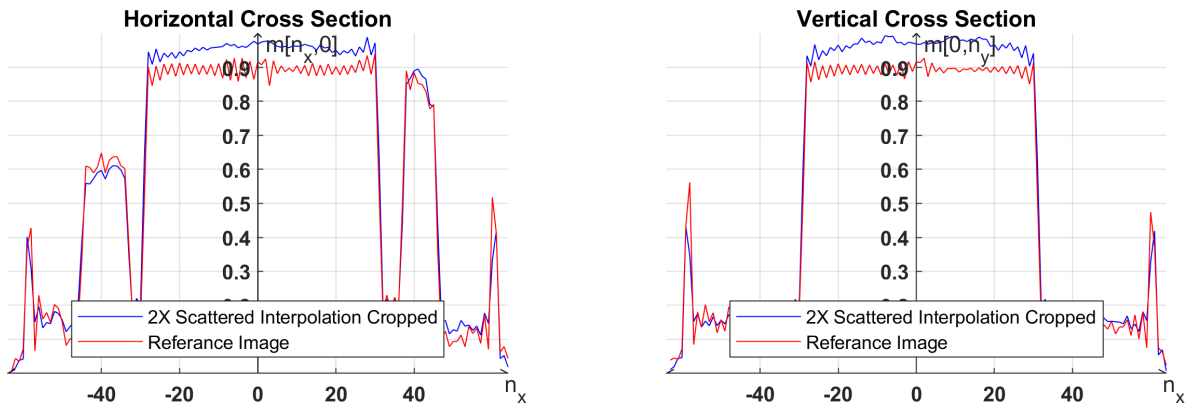


Figure 1.12.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 24.5874, SSIM = 0.7668, Processing Time = 0.0017 seconds, CPU Time = 0.0781.

```

1  t0 = cputime;
2  time_elapsed9 = cputime-t0
3  ima_interp_2_full = scattered_interpolate(spiral_data.ktraj,spiral_data.kdata,256);
4  ima_interp_2_crop = ima_interp_2_full(65:192, 65:192);
5  ima_interp_2_crop = ima_interp_2_crop / max(ima_interp_2_crop(:));
6
7  figure;
8  imshow(ima_interp_2_full)
9  title("2X Interpolated Image")
10 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
11 if save_image
12     exportgraphics(gcf, strcat("q112_image.png"), 'Resolution', 300)
13 end
14 plot_comparative_images(ima_interp_2_crop, ima_direct, save_image, "q112_comp", '2X
    ↳ Scattered Interpolation Cropped')
15 plot_cross_sections(ima_interp_2_crop, ima_direct, save_image, "q112_cross", '2X Scattered
    ↳ Interpolation Cropped')
16 PSNR9 = psnr(abs(ima_interp_2_crop), ima_direct)
17 SSIM9 = ssim(abs(ima_interp_2_crop), ima_direct)

```

Question 2

In this question, the *Modified Shepp-Logan* phantom was generated in MATLAB and used to implement various methods. We will take the projection of the phantom from 180 different angles and reconstruct from this data using filtered backprojection and *naïve* backprojection. After these methods, we will use the central slice theorem to convert the projection data into k-space data, which we will reconstruct using direct summation and gridding. We will again compare these methods based on SSIM, PSNR, and computation time.

Question 2.1

We can see the phantom, which will be used as the reference image for the following questions, in Figure 2.1.1. We also take the projections of this image from multiple angles using the Radon transform. The result of this transform is a sinogram visible in Figure 2.1.3.

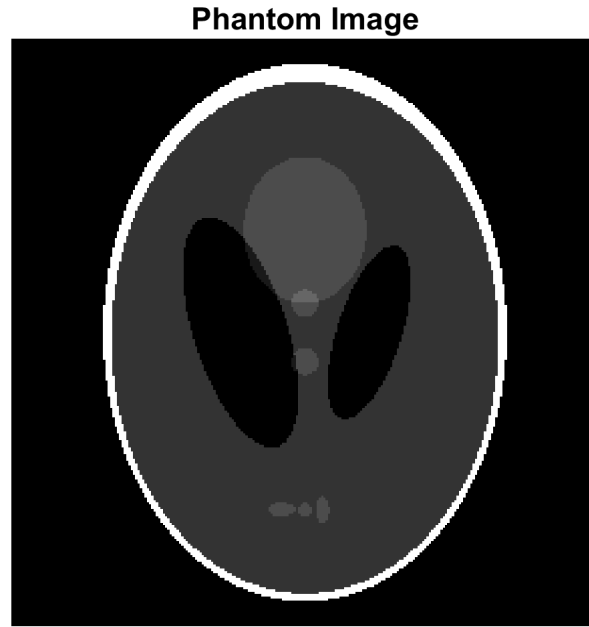


Figure 2.1.1: Modified Shepp-Logan Phantom

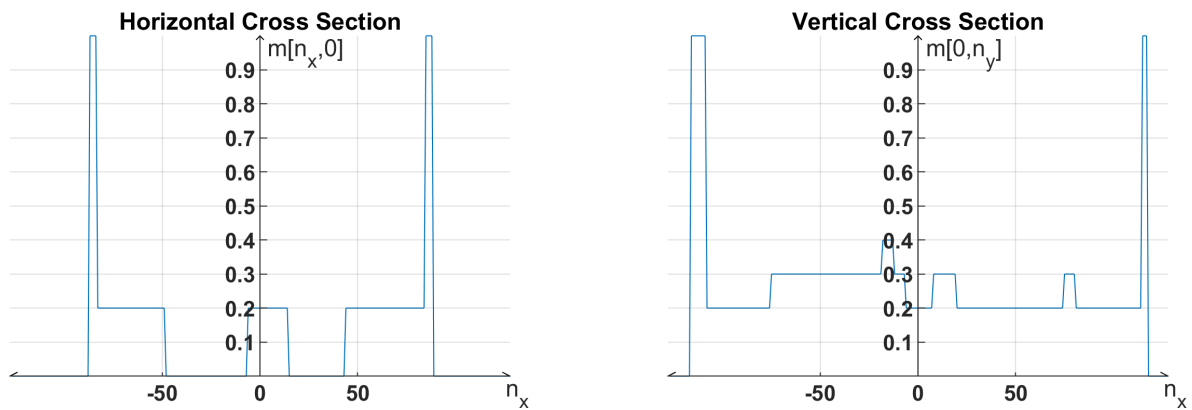


Figure 2.1.2: Vertical and Horizontal Cross-Sections of the Image

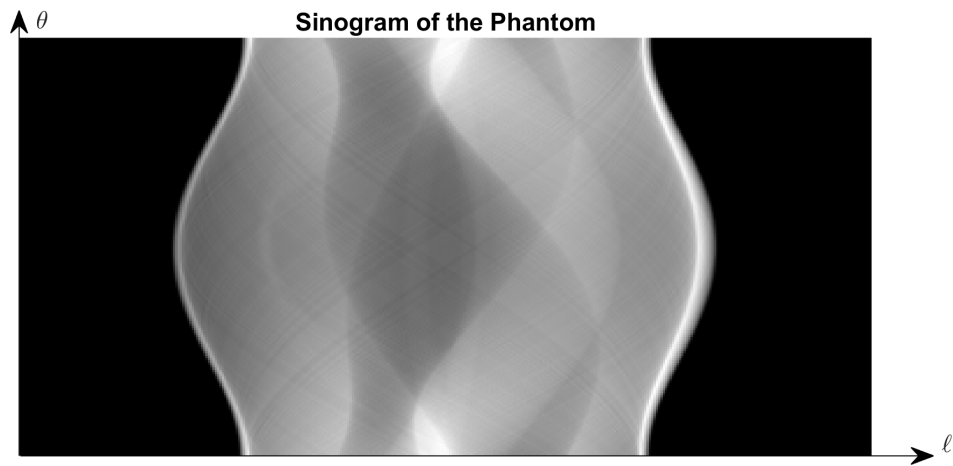


Figure 2.1.3: Sinogram of the Phantom

```

1  P = phantom('Modified Shepp-Logan',256);
2  P = P/max(P(:));
3  clf;
4  imshow(P)
5  title("Phantom Image")
6  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
7  if save_image
8      exportgraphics(gcf, strcat("q21_image.png"), 'Resolution', 300)
9  end
10 subplot(1,2,1)
11 fineplot(1:256,abs(P(end/2,:)), "Horizontal Cross Section", "n_x", "m[n_x,0]", [-N/2
    ↪ N/2], [0 1], "on", [800 400], "Reference Image", "-");
12 grid on
13 subplot(1,2,2)
14 fineplot(1:256,abs(P(:,end/2)), "Vertical Cross Section", "n_x", "m[0,n_y]", [-N/2 N/2], [0
    ↪ 1], "on", [800 400], "Reference Image", "-");
15 grid on
16 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
17 if save_image
18     exportgraphics(gcf, strcat("q21_cross_sections.png"), 'Resolution', 300)
19 end
20 proj = radon(P,0:179);
21 clf;
22 imshow(proj.', [])
23 xlabel("$\ell$", "Interpreter", "latex", "Position", [400 170])
24 ylabel("$\theta$", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)
25 title("Sinogram of the Phantom")
26
27 annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
28 annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
29 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
30 if save_image
31     exportgraphics(gcf, strcat("q21_sinogram.png"), 'Resolution', 300)
32 end

```

Question 2.2

Using the inverse Radon transform in MATLAB, we can obtain the filtered backprojection of the sinogram using the ramp filter discussed in class. This filtering process compensates for the density and produces an accurate image.

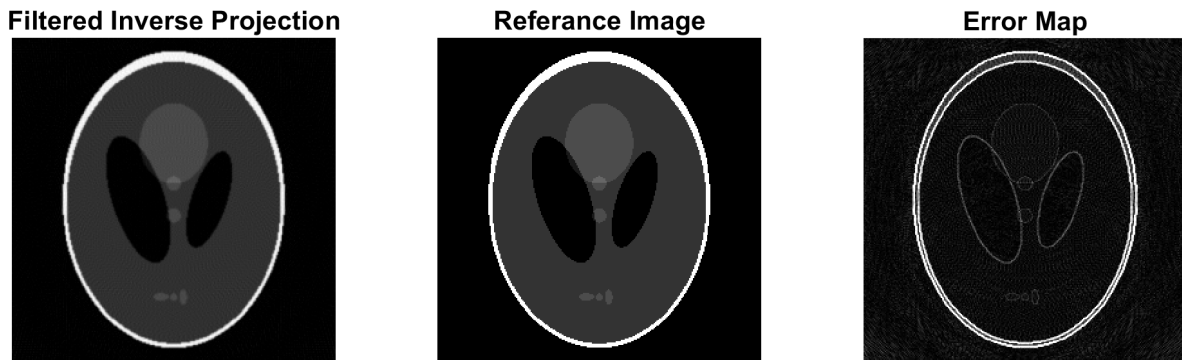


Figure 2.2.1: Comparison of Filtered Backprojected Image with the Reference Image

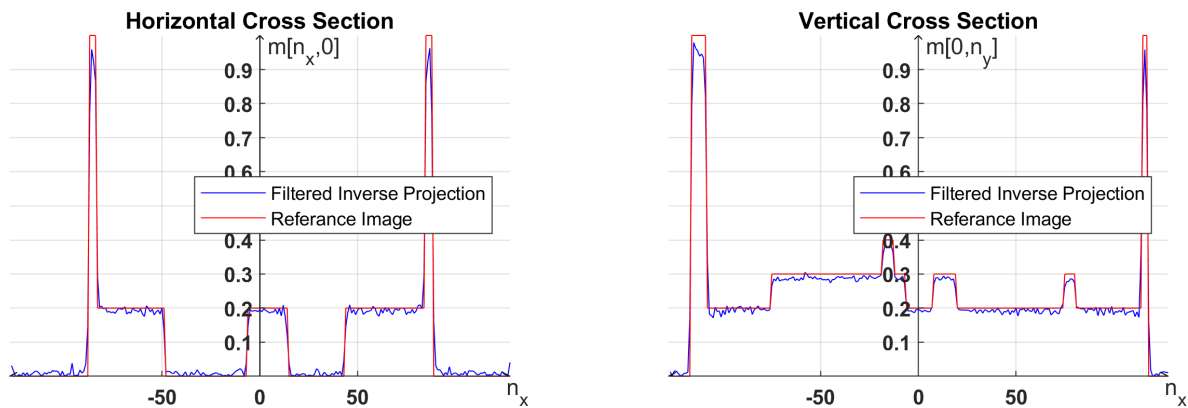


Figure 2.2.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 26.8242, SSIM = 0.6799, Processing Time = 0.0041 seconds, CPU Time = 0.

```

1 inverse_proj = iradon(proj,0:179);
2 t0 = cputime;
3 inverse_proj = inverse_proj(2:257,2:257);
4 time_elapsed2_1 = t0 - cputime
5 inverse_proj = inverse_proj/max(inverse_proj(:));
6 plot_comparative_images(inverse_proj,P,save_image,"q22_comp","Filtered Inverse
  ↳ Projection")
7 plot_cross_sections(inverse_proj,P,save_image,"q22_cross","Filtered Inverse
  ↳ Projection","Reference Image","west")
8 PSNR2_1 = psnr(abs(inverse_proj),P)
9 SSIM2_1 = ssim(abs(inverse_proj),P)

```

Question 2.3

We can again use the inverse Radon transform without using the density compensation filter to obtain the direct backprojection, which will be referred to as the *naive backprojection* throughout the assignment. Similar to question 1.5, since we are not compensating for the density of low-frequency components, the result is a hazy, blurry image. Since the error is significant for this question, the error for this question was plotted in the interval $[0, 1]$ in addition to the standard $[0, 0.2]$. Also, we can notice that the cross sections do not overlap properly. The compensated versions are available in Section A.3



Figure 2.3.1: Comparison of Naive Backprojected Image with the Reference Image

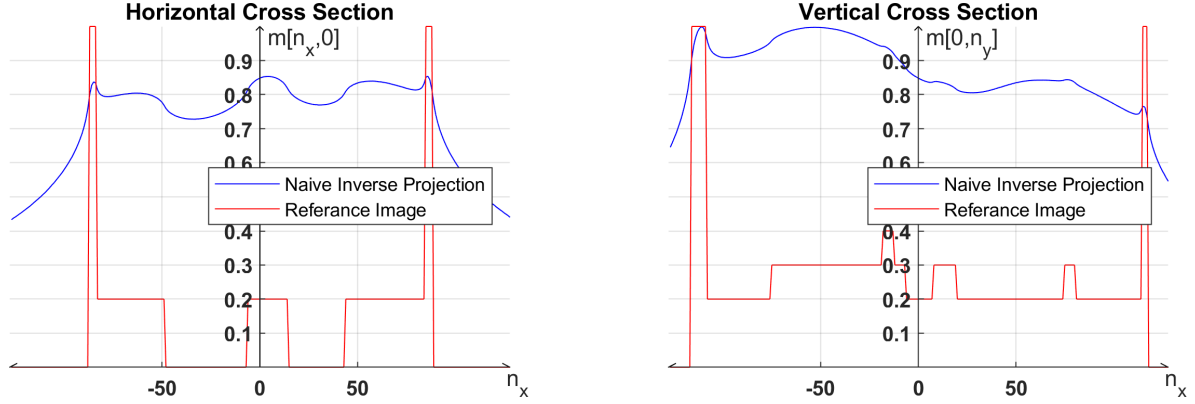


Figure 2.3.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 4.7234, SSIM = 0.1535, Processing Time = 0.0165 seconds, CPU Time = 0.0156.

```

1  t0 = cputime;
2  naive_proj = iradon(proj,0:179,"None");
3  time_elapsed2_2 = t0 - cputime;
4  naive_proj = naive_proj(2:257,2:257);
5  naive_proj = naive_proj/max(naive_proj(:));
6  % naive_proj = naive_proj * 0.25; % for normalisation correction. comment to disable
7  plot_comparative_images(naive_proj,P,save_image,"q23_comp","Naive Inverse
   ↳ Projection","Reference Image",[0 1])
8  plot_cross_sections(naive_proj,P,save_image,"q23_cross","Naive Inverse
   ↳ Projection","Reference Image","north")
9  PSNR2_2 = psnr(abs(naive_proj),P)
10 SSIM2_2 = ssim(abs(naive_proj),P)

```

Question 2.4

In order to obtain the k-space equivalent of the data, taking the 1D Fourier Transform of the sinogram along ℓ direction results in radial lines in k-space according to the central slice theorem [5]. We then *fftshift* these data in order to align the DC components with the center of the radial lines. We generate rotated lines in k-space for each line of data with a uniform angle distribution. The result of this process is 180 uniformly spaced lines in k-space, each with its respective data.

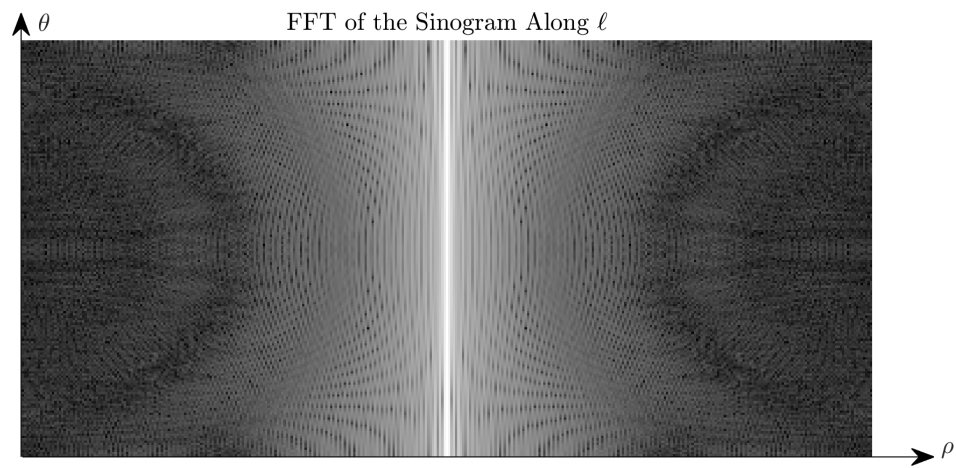


Figure 2.4.1: 1DFFT of the Sinogram (K-Space Data)



Figure 2.4.2: Radial Trajectory of the Projection Data

```

1  clf;
2  kdata = fftshift(fft(iffshift(proj,1), [], 1), 1);
3  imshow(log10(1+abs(kdata.')),[])
4  title("FFT of the Sinogram Along $\ell$", "Interpreter", "latex")
5  xlabel("$\rho$", "Interpreter", "latex", "Position", [400 170])
6  ylabel("$\theta$", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)
7
8  annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
9  annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
10 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
11 if save_image
12     exportgraphics(gcf, strcat("q24_sinogram_fft.png"), 'Resolution', 300)
13 end
14 angles = [0:179]*pi/180;
15 radii = linspace(-0.5, 0.5, 367);
16 ktraj = radii.' * exp(1j*angles);
17 fineplot(real(ktraj), imag(ktraj), "Radial Trajectories", "k_x", "k_y", [-0.55 0.55], [-0.55
    ↪ 0.55], "off", [400 400], "Trajectory", "-");
18 grid on
19 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.85]);
20 if save_image
21     exportgraphics(gcf, strcat("q24_trajectory.png"), 'Resolution', 300)
22 end

```

Question 2.5

The area corresponding to a sample can be written with the formula:

$$d_n = \frac{2\pi}{N}(\Delta k)^2 |n| \quad \forall n \neq 0, d_0 = \frac{2\pi}{4N}(\Delta k)^2 \quad (2.5.1)$$

where d_n denotes the density of the n 'th circle, N denotes the number of lines, Δk denotes the gap between the consecutive samples on the same line.

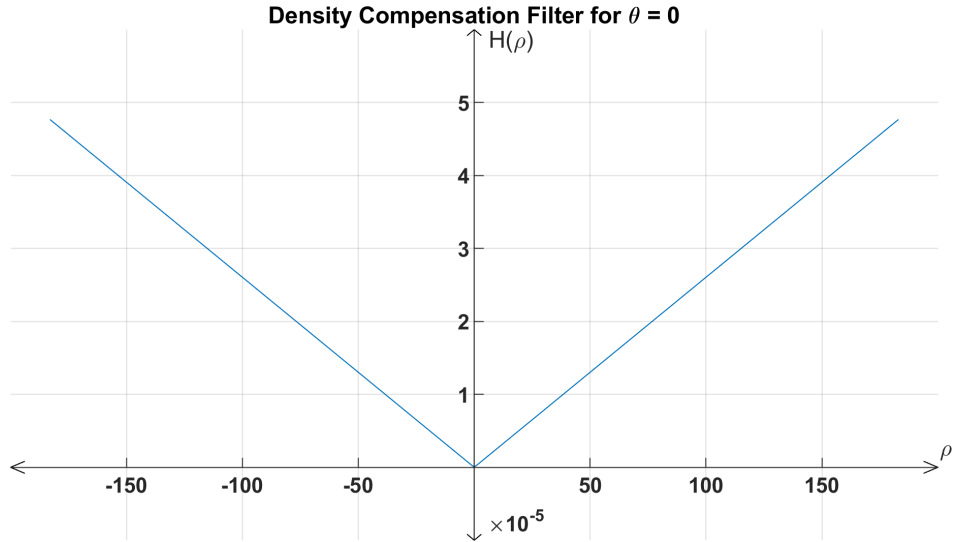


Figure 2.5.1: Geometrically Calculated Density Compensation Filter

```

1  N = 180;
2  delta_k = 1/366;
3  n = 0:(367-1)/2;
4  d = pi/N*2*(delta_k^2)*n;
5  d(1) = pi/(4*N)*((delta_k)^2);
6  d = [flip(d) d(2:end)];
7  d = meshgrid(d,1:180);
8  area3 = d;
9  clf;
10 fineplot(-366/2:366/2,d(1,:), "Density Compensation Filter for \theta =
    ↪ 0", "\rho", "H(\rho)", [-200 200], [-0.00001 0.00006], "off", [400 400], "", "-");
11 grid on
12 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
13 if save_image
14     exportgraphics(gcf, strcat("q25_dcf.png"), 'Resolution', 300)
15 end

```

Question 2.6

Using the direct summation method, we can reconstruct the image from the generated k-space data. This method again takes significantly longer computation time compared to the other methods and provides decent times.

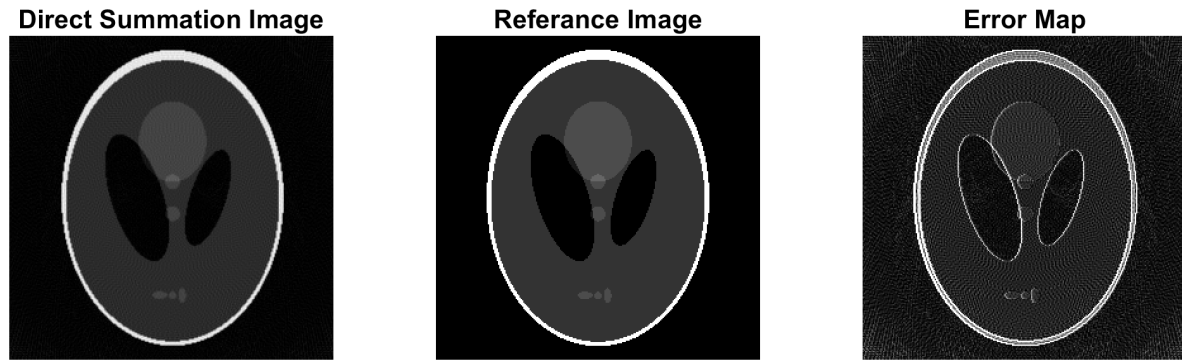


Figure 2.6.1: Comparison of Direct Summation Image with the Reference Image

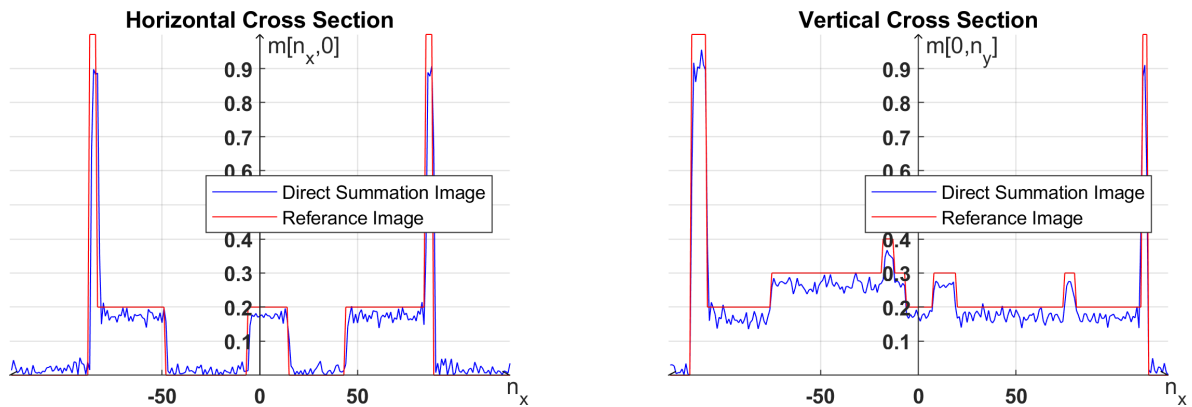


Figure 2.6.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 20.3011, SSIM = 0.4669, Processing Time = 64.4836 seconds, CPU Time = 461.6562.

```

1  t0 = cputime;
2  ima_direct = direct_summation(ktraj,kdata,d.',256,1);
3  time_elapsed2_3 = t0 - cputime
4  ima_direct = flipud(ima_direct);
5  plot_comparative_images(ima_direct,P,save_image,"q26_comp","Direct Summation Image")
6  plot_cross_sections(ima_direct,P,save_image,"q26_cross","Direct Summation Image")
7  PSNR2_3 = psnr(abs(ima_direct),P/max(P(:)))
8  SSIM2_3 = ssim(abs(ima_direct),P/max(P(:)))

```

Question 2.7

We also reconstruct the non-Cartesian radial k-data using 2X gridding. This method offers higher performance compared to the direct summation method while still significantly behind filtered back-projection. Again, the normalisation compensated versions are available in Section A.4

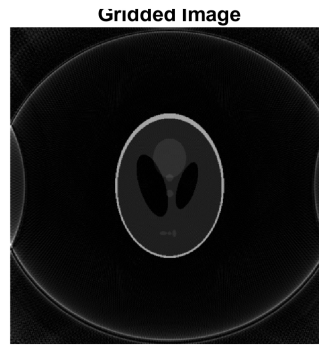


Figure 2.7.1: 2X Gridded Image



Figure 2.7.2: Comparison of 2X Gridded Cropped Image with the Reference Image

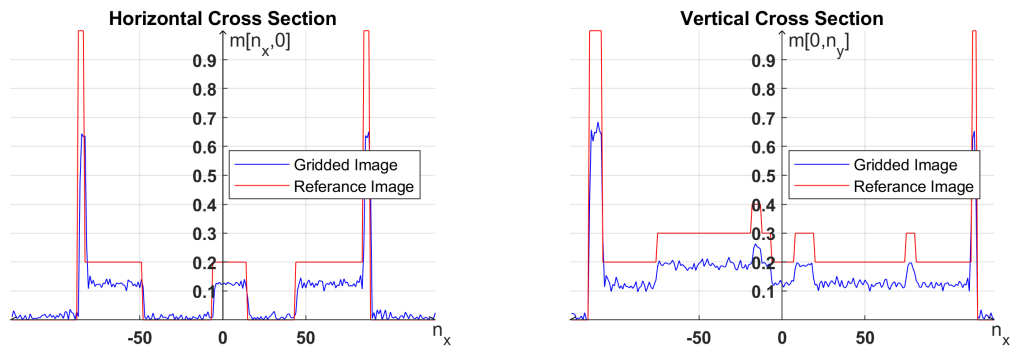


Figure 2.7.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 18.3281, SSIM = 0.5210, Processing Time = 0.2630 seconds, CPU Time = 0.5.


```

1 t0 = cputime;
2 ima_gridding = fliplr(gridkb(kdata,ktraj,area3.',256,2,4,'image'));
3 time_elapsed2_4 = t0 - cputime
4 ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
5 % ima_gridding = ima_gridding * (1/0.7); % for normalisation correction. comment to disable
6 clf;
7 imshow(ima_gridding)
8 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
9 if save_image
10     exportgraphics(gcf, strcat("q27_image.png"), 'Resolution', 300)
11 end
12 plot_comparative_images(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_comp","Gridded
    ↪ Image")
13 plot_cross_sections(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_cross","Gridded
    ↪ Image")
14 PSNR2_4 = psnr(abs(ima_gridding(129:512-128,129:512-128)),P)
15 SSIM2_4 = ssim(abs(ima_gridding(129:512-128,129:512-128)),P)

```

Question 3

In this question, we are given k-space data corresponding to the *Modified Shepp-Logan* phantom.

Question 3.1

In this question, unlike question 2, the radial lines start from the origin, as is usually the case in MRI. There are 300 lines, with each having 129 samples.

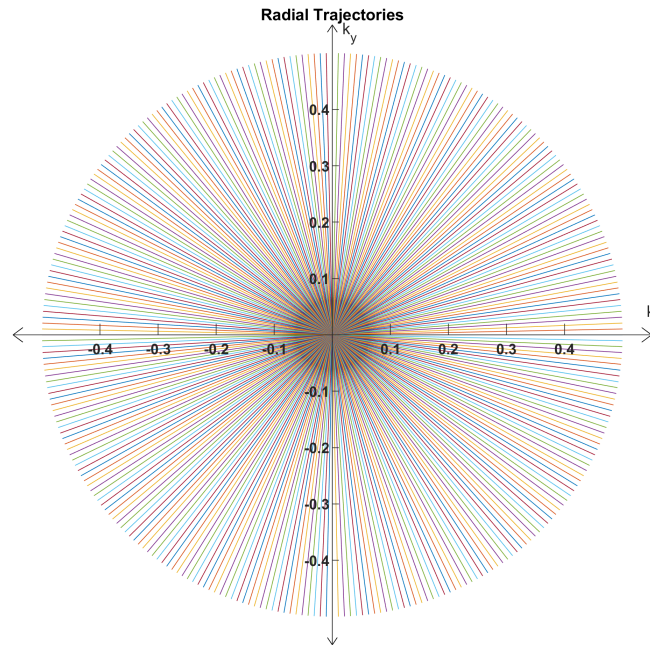


Figure 3.1.1: Radial K-Space Trajectory

```

1 radial_data = load("shepplogan_radial_data.mat");
2 clf;
3 fineplot(real(radial_data.ktraj),imag(radial_data.ktraj),"Radial
    ↪ Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[400 400],"-", "-");
4 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.85]);
5 if save_image
6     exportgraphics(gcf, strcat("q31_trajectory.png"), 'Resolution', 300)
7 end

```

Question 3.2

Using the same theoretical foundation as in question 2.5, we can calculate the density compensation filter. Due to the differences in k-space trajectories, we only have a single side of the ramp function as the lines start from the origin again, as opposed to question 2. This consequently results in PSNR and SSIM values much lower than those the method actually achieved on the phantom itself.

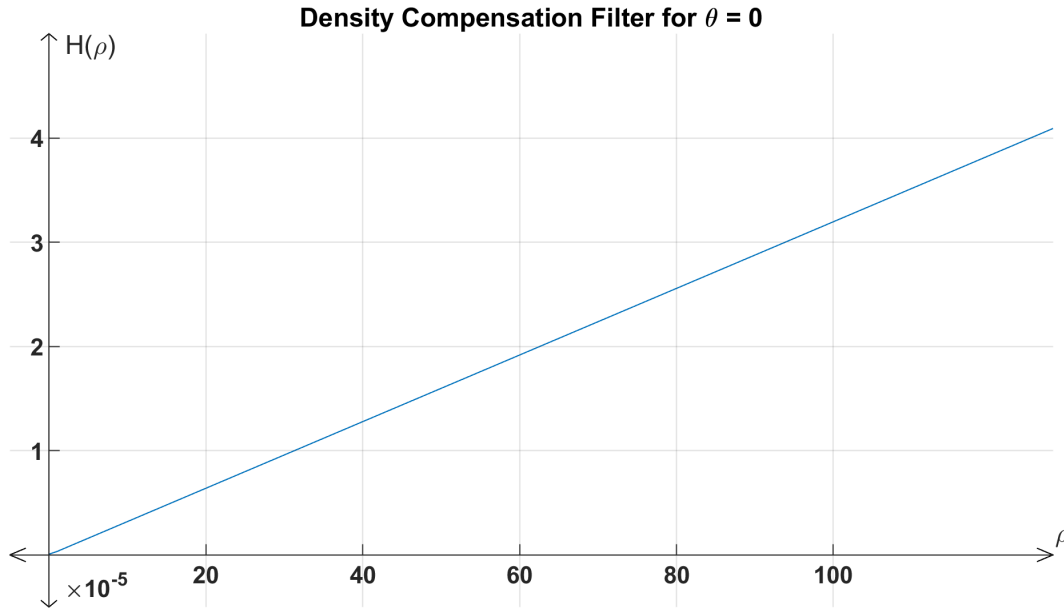


Figure 3.2.1: Geometrically Calculated Density Compensation Filter

```

1 d = [0:128]*(pi/300)*((0.5/128)^2)*2;
2 d(1) = pi/(4*N)*((0.5/128)^2);
3 clf;
4 figure(0:128,d,"Density Compensation Filter for \theta = 0","\rho","H(\rho)",[-5
   ↪ 128],[-0.000005 0.00005],"off",[400 400],"","-");
5 grid on
6 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
7 if save_image
8     exportgraphics(gcf, strcat("q32_dcf.png"), 'Resolution', 300)
9 end
10 d = meshgrid(d,1:300);
11 area4 = d;

```

Question 3.3

After obtaining the k-space data, we can reconstruct the image using the methods we used previously. Using the direct summation approach provides decent results, but we see significant artifacts in the image. This is because we decreased the number of samples per line while keeping k_{extd} constant. We had 167 samples in the interval $[0, 0.5]$ in question 2, whereas we only have 129 samples in the same interval. This increase in Δk results in a decrease in the FOV. Therefore, the rings we see in Figure ?? are aliased closer to the actual object due to the decrease in FOV.

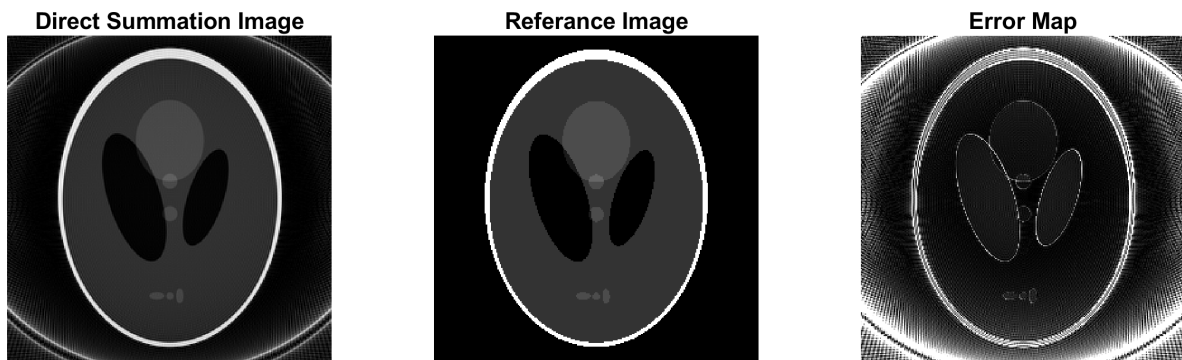


Figure 3.3.1: Comparison of Direct Summation Image with the Reference Image

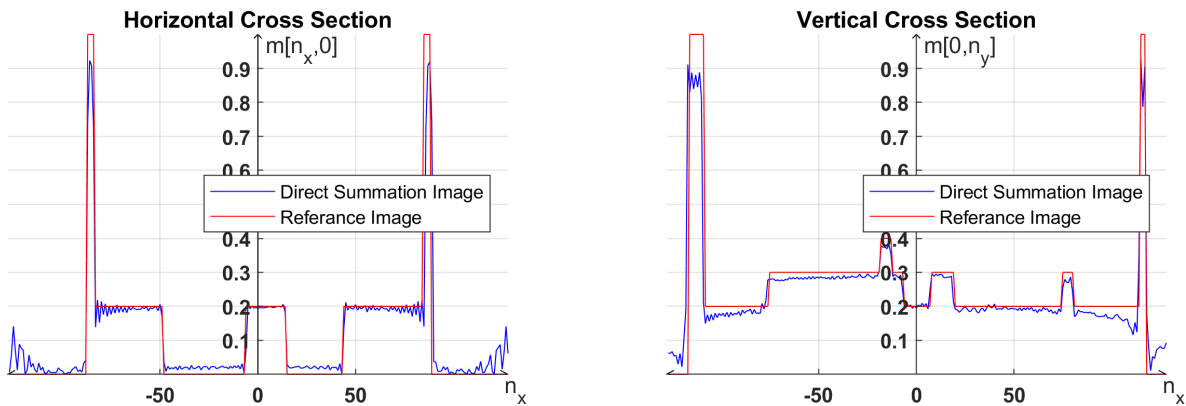


Figure 3.3.2: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 18.2282, SSIM = 0.4635, Processing Time = 31.0797 seconds, CPU Time = 231.6094.

```

1  t0 = cputime;
2  ima_direct = direct_summation(radial_data.ktraj,radial_data.kdata,d.',256,1);
3  time_elapsed3_1 = t0 - cputime
4  ima_direct = flipud(ima_direct);
5  plot_comparative_images(ima_direct,P,save_image,"q33_comp","Direct Summation Image")
6  plot_cross_sections(ima_direct,P,save_image,"q33_cross","Direct Summation
   ↪ Image","Reference Image","north")
7  PSNR3_2 = psnr(abs(ima_direct),P)
8  SSIM3_2 = ssim(abs(ima_direct),P)

```

Question 3.4

We see the same results as question 3.3. The artifacts are visible outside the previously visible FOV. The PSNR and SSIM values suffer from these artifacts again.

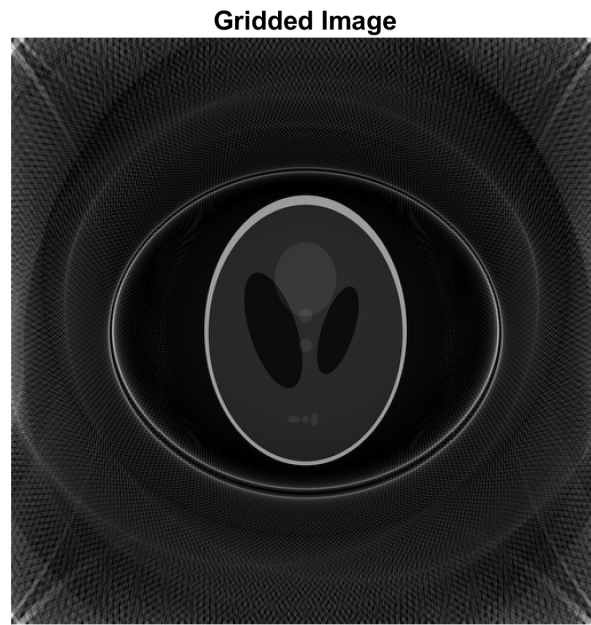


Figure 3.4.1: 2X Gridded Image

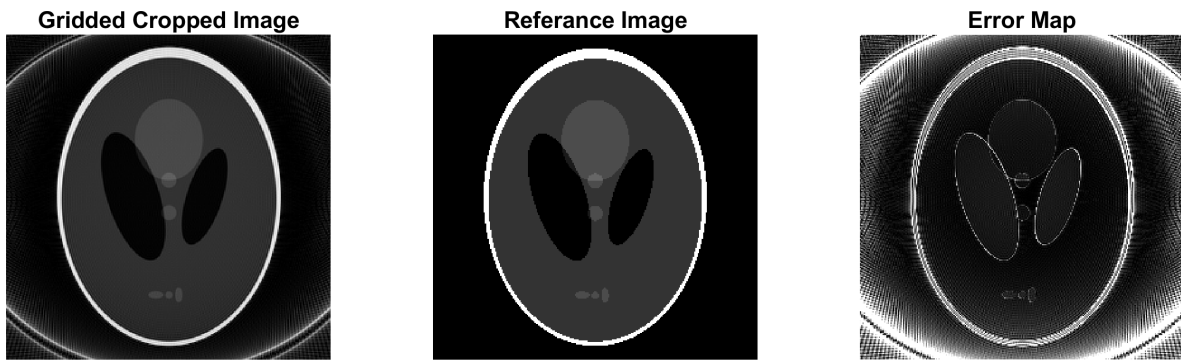


Figure 3.4.2: Comparison of 2X Gridded Cropped Image with the Reference Image

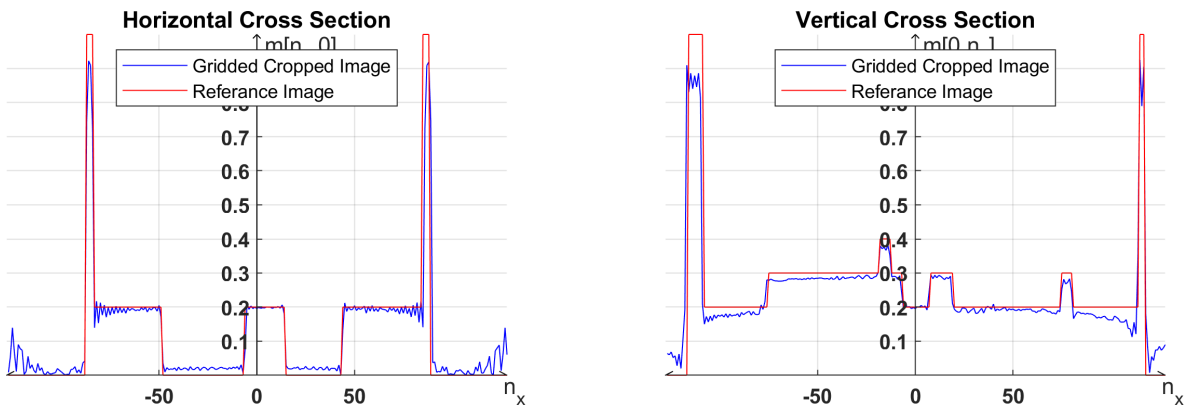


Figure 3.4.3: Vertical and Horizontal Cross-Section Comparisons of the Images

PSNR = 18.2307, SSIM = 0.4636, Processing Time = 0.1174 seconds, CPU Time = 0.3906.

```

1 t0 = cputime;
2 ima_gridding = fliplr(gridkb(radial_data.kdata,radial_data.ktraj,area4.',256,2,4,'image'));
3 time_elapsed3_2 = t0 - cputime
4 ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
5 clf;
6 imshow(ima_gridding)
7 title("Gridded Image")
8 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
9 if save_image
10     exportgraphics(gcf,strcat("q34_image.png"),'Resolution',300)
11 end
12 ima_gridding_cropped = ima_gridding(129:512-128,129:512-128);
13 ima_gridding_cropped = ima_gridding_cropped/max(ima_gridding_cropped(:));
14 plot_comparative_images(ima_gridding_cropped,P,save_image,"q34_comp","Gridded Cropped Image")
15 plot_cross_sections(ima_gridding_cropped,P,save_image,"q34_cross","Gridded Cropped
    ↪ Image","Referance Image","north")
16 PSNR3_2 = psnr(abs(ima_gridding_cropped),P)
17 SSIM3_2 = ssim(abs(ima_gridding_cropped),P)

```

Conclusion

In this assignment, multiple reconstruction methods for non-Cartesian data were used. Mainly, direct summation, gridding, and interpolation were used on spiral and radial k-space data along with projection reconstruction.

For spiral data, direct summation provided most accurate results while taking the most time. Gridding was found to be a decent tradeoff between accuracy and processing time. Interpolation was the quickest method even though it lacked the same accuracy as the other methods.

For projection data, filtered backprojection provided the most accurate data while beating direct summation in terms of processing time aswell.

Overall, this assignment provided a clear overview of different non-Cartesian reconstruction methods as well as their evaluations and comparisons.

References

- [1] "Spiral k-space dataset." MATLAB .mat file provided with EEE 475 Homework 2, 2026.
- [2] "Shepp-logan radial k-space dataset." MATLAB .mat file provided with EEE 475 Homework 2, 2026.
- [3] "voronoidens.m: Voronoi density compensation function." MATLAB script provided with EEE 475 Homework 2, 2026.
- [4] "gridkb.m: Gridding reconstruction function." MATLAB script provided with EEE 475 Homework 2, 2026.
- [5] D. G. Nishimura, *Principles of Magnetic Resonance Imaging*. Stanford University, 2010.

Appendix A: Images with Normalisation Correction

In some scenarios, due to local fluctuations, normalisation may result in incorrect scaling of images. The images where the cross sections were adjusted to overlap by scaling with a constant were also calculated as a comparison to the solely normalised images.

Question 1.8

The image was scaled by a factor of 1.2.

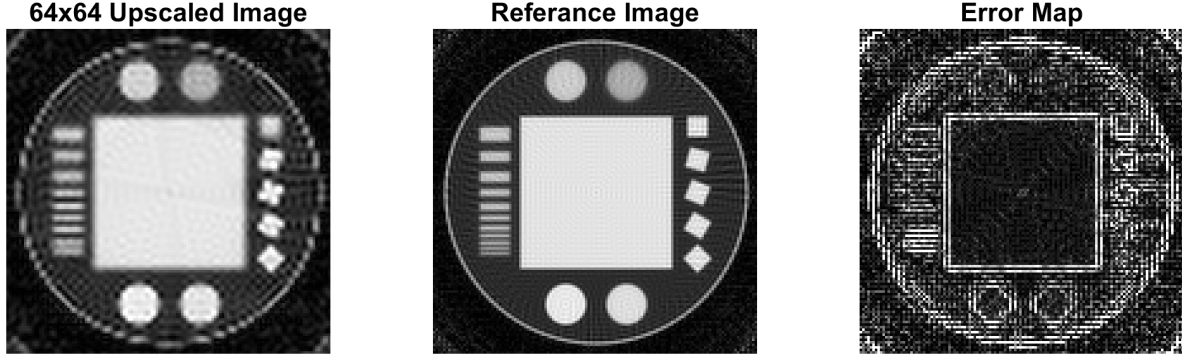


Figure A.1.1: Comparison of Normalisation Corrected Upscaled 64x64 Image with the Reference Image

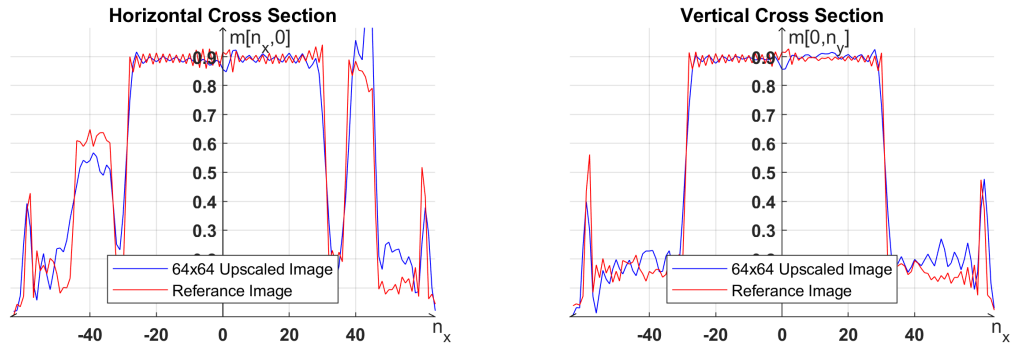


Figure A.1.2: Vertical and Horizontal Cross-Section Comparisons of the Images

$$\text{PSNR} = 21.6302, \text{SSIM} = 0.6535$$

Question 1.9

The image was scaled by a factor of 1.08.

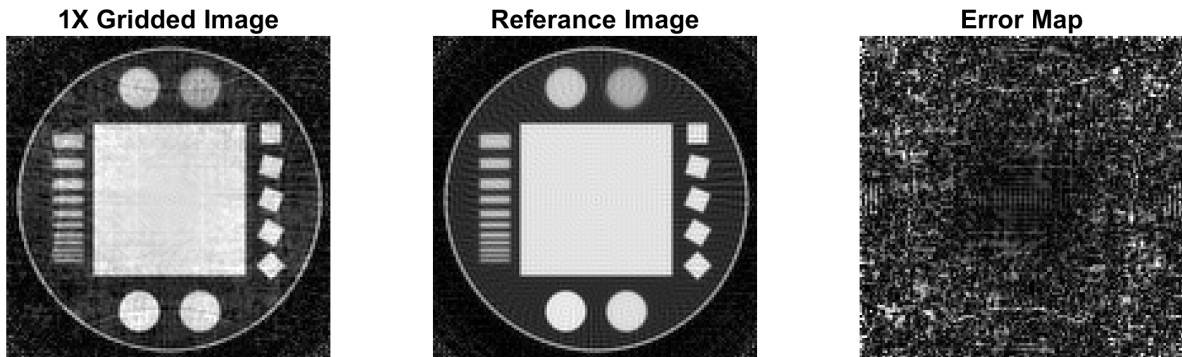


Figure A.2.1: Comparison of Normalisation Corrected 1X Gridded Image with the Reference Image

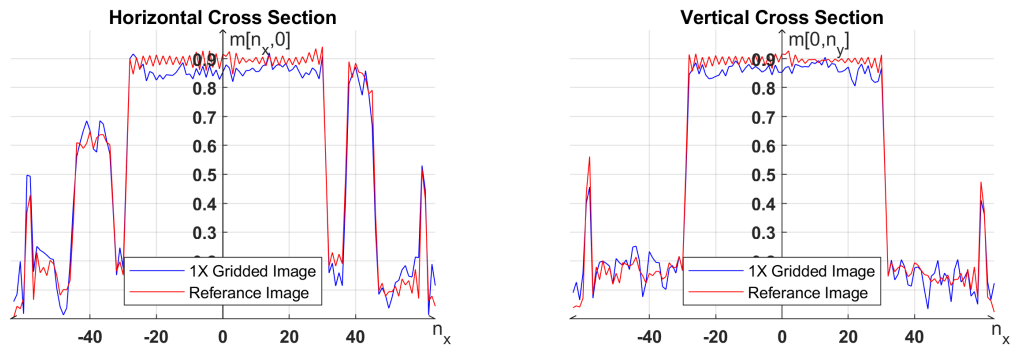


Figure A.2.2: Vertical and Horizontal Cross-Section Comparisons of the Images

$$\text{PSNR} = 25.3083, \text{SSIM} = 0.7461$$

Question 2.3

The image was scaled by a factor of 0.25.

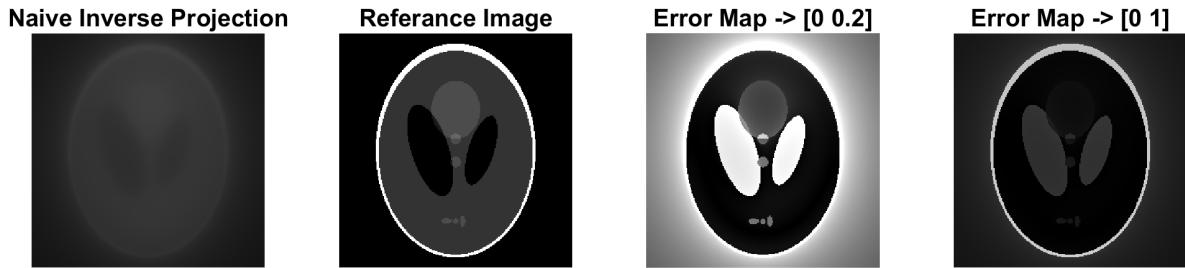


Figure A.3.1: Comparison of Normalisation Corrected Naive Backprojected Image with the Reference Image

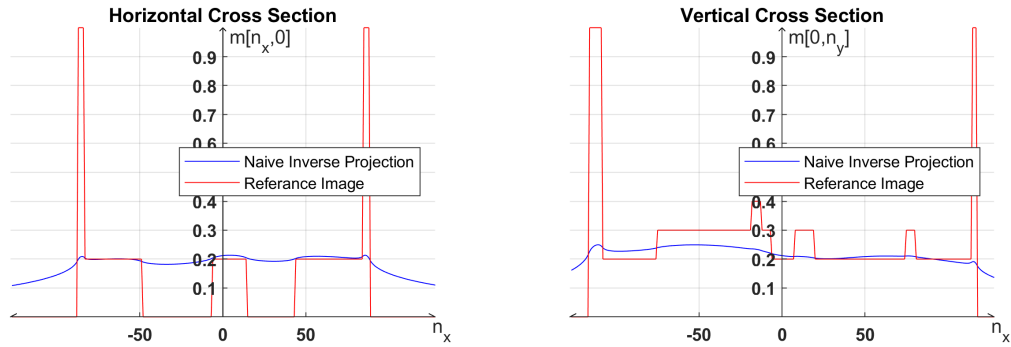


Figure A.3.2: Vertical and Horizontal Cross-Section Comparisons of the Images

$$\text{PSNR} = 14.1536, \text{SSIM} = 0.3101$$

Question 2.7

The image was scaled by a factor of 1.42.

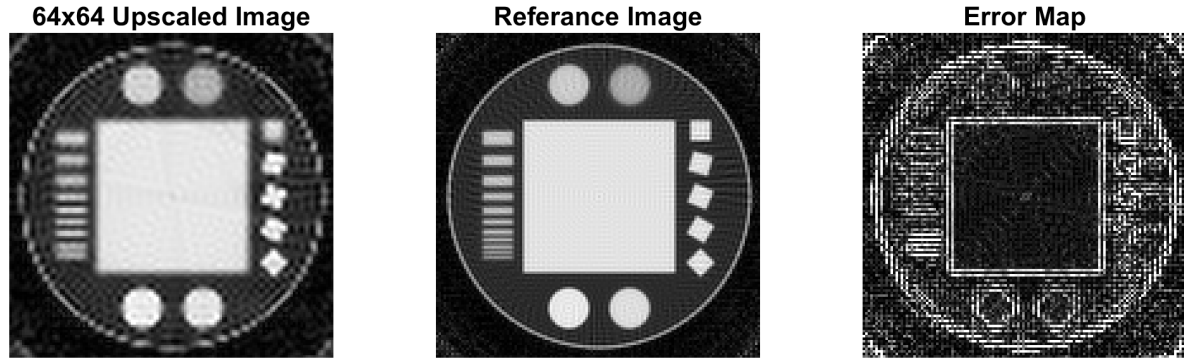


Figure A.4.1: Comparison of Normalisation Corrected 2X Gridded Cropped Image with the Reference Image

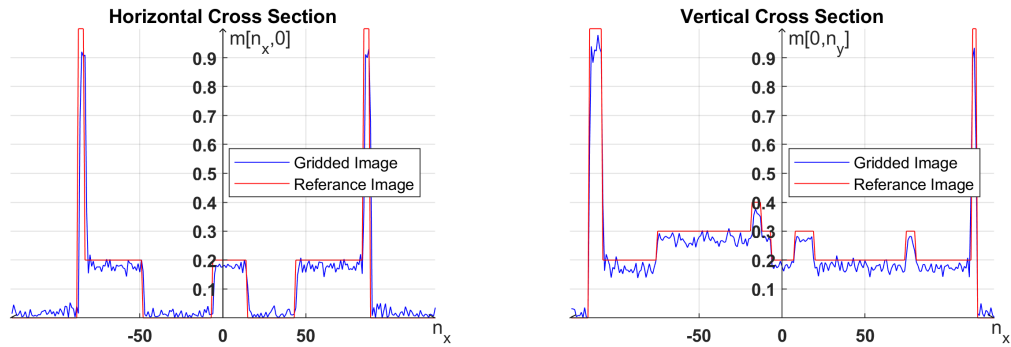


Figure A.4.2: Vertical and Horizontal Cross-Section Comparisons of the Images

$$\text{PSNR} = 20.5299, \text{SSIM} = 0.4874$$

Comments on Normalisation Correcting

Even though cross sections overlap with each other more closely, the SSIM and PSNR values both degraded for most cases. It does not seem to generally improve results.

Questions	1.8	1.9	2.3	2.7
Default SSIM Values	0.6682	0.7482	0.1535	0.5210
Corrected SSIM Values	0.6535	0.7461	0.3101	0.4616

Table A.5.1: SSIM Value Comparisons for Normalisation Correction

Questions	1.8	1.9	2.3	2.7
Default PSNR Values	21.6566	25.3592	4.7234	18.3281
Corrected PSNR Values	21.6302	25.3083	14.1536	20.3606

Table A.5.2: PSNR Value Comparisons for Normalisation Correction

Appendix B: MATLAB Script and Functions

```
save_image = 0
```

Question 1.1

```
spiral_data = load("spiral.mat");
figure;
fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"K-Space
Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[600 900],"","-");
grid on

set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q11_trajectory.png"),'Resolution',300)
end
```

Question 1.2

```
area = voronoidens(spiral_data.ktraj);
total_nans = sum(isnan(area), 'all')
subplot(1,2,1)
fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample","Area",[-500
length(area(:))],[-0.02 0.1],"off",[800 400],"","-");
grid on
subplot(1,2,2)
fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample","Area",[-500
length(area(:))],[-0.00004 0.0002],"off",[800 400],"","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q12_area.png"),'Resolution',300)
end
```

Question 1.3

```
figure;
circle = 0.51*exp(1*j*linspace(0,2*pi,2049));
circle = circle(1:end-1);
limited_ktraj=[spiral_data.ktraj circle.'];
fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"Boundary and K-
Space Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"on",[400
800],"Trajectory","-");
plot(circle,"LineWidth",5,'displayName',"Boundary")
legend()
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q13_trajectory.png"),'Resolution',300)
end
figure;
area2 = voronoidens(limited_ktraj);
area2 = area2(:,1:6);
subplot(1,3,1:2)
fineplot(1:length(area2(:)),area2(:),"Area of Each Cell","#sample","Area",[-500
length(area2(:))],[-0.00004 0.0002],"off",[800 400],"","-");
```



```

grid on
subplot(1,3,3)
fineplot(1:length(area2(:,1)),area2(:,1),"Area of Single
Trajectory","#sample","Area",[-500 length(area2(:,1))],[-0.00004 0.0002],"off",
[800 400],"","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q13_area.png"), 'Resolution', 300)
end

```

Question 1.4

```

N=128;
t0 = cputime;
ima_direct = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,128,1);
time_elapsed1 = cputime-t0
figure;
imshow(ima_direct)
title("Reference Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q14_image.png"), 'Resolution', 300)
end
figure;
subplot(1,2,1)
fineplot([1:N]-N/2,abs(ima_direct(end/2,:)),"Horizontal Cross
Section","n_x","m[n_x,0]",[-N/2 N/2],[0 1],"off",[800 400],"","-");
grid on
subplot(1,2,2)
fineplot([1:N]-N/2,abs(ima_direct(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y]",[-N/2 N/2],[0 1],"off",[800 400],"","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q14_cross.png"), 'Resolution', 300)
end

```

Question 1.5

```

N=128;
t0 = cputime; tic()
ima_direct2 =
direct_summation(spiral_data.ktraj,spiral_data.kdata,ones(size(spiral_data.kdata)
),128,1);
time_elapsed2 = cputime-t0
plot_comparative_images(ima_direct2,ima_direct,save_image,"q15_comp","No Density
Compensation","Reference Image",[0 1])
plot_cross_sections(ima_direct2,ima_direct,save_image,"q15_cross","No Density
Compensation")
PSNR2 = psnr(abs(ima_direct2),ima_direct)
SSIM2 = ssim(abs(ima_direct2),ima_direct)

```

Question 1.6

```
N3 = 256;
t0 = cputime;
ima_direct3 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,256,1);
time_elapsed3 = cputime-t0
cropped = ima_direct3(N3/4+1:3*N3/4,N3/4+1:3*N3/4);
figure;
imshow(ima_direct3)
title("256 x 256 Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q16_image.png"), 'Resolution', 300)
end
plot_comparative_images(cropped,ima_direct,save_image,"q16_comp","256x256 Direct
Summed Cropped Image")
plot_cross_sections(cropped,ima_direct,save_image,"q16_cross","256x256 Direct
Summed Cropped Image")
PSNR3 = psnr(abs(cropped),ima_direct)
SSIM3 = ssim(abs(cropped),ima_direct)
```

Question 1.7

```
N4 = 256;
t0 = cputime;
ima_direct4 =
direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,256,0.5);
time_elapsed4 = cputime-t0
k_ref = fft2c(ima_direct);
k_ref_padded = zeros([N4 N4]);
k_ref_padded(N4/4+1:(3*N4/4),N4/4+1:(3*N4/4)) = k_ref;
ima_direct_us = ifft2c(k_ref_padded);
ima_direct_us = ima_direct_us/max(ima_direct_us(:));
plot_comparative_images(abs(ima_direct4),abs(ima_direct_us),save_image,"q17_comp"
,"256x256 Direct Summed Image","Upscaled Reference Image")
plot_cross_sections(ima_direct4,ima_direct_us,save_image,"q17_cross","256x256
Direct Summed Image","Upscaled Reference Image")
PSNR4 = psnr(abs(ima_direct4),abs(ima_direct_us))
SSIM4 = ssim(abs(ima_direct4),abs(ima_direct_us))
```

Question 1.8

```
t0 = cputime;
ima_direct5 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,64,2);
time_elapsed5 = cputime-t0
k_data_5 = fft2c(ima_direct5);
k_data_5_padded = zeros([N,N]);
k_data_5_padded(N/4+1:3*N/4,N/4+1:3*N/4) = k_data_5;
ima_direct5_us = ifft2c(k_data_5_padded);
ima_direct5_us = abs(ima_direct5_us)/max(abs(ima_direct5_us(:)));
% ima_direct5_us = ima_direct5_us * 1.2; %normalisation correction? comment to
disable
figure;
imshow(ima_direct5)
```

```

title("64x64 Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q18_image.png"), 'Resolution', 300)
end
plot_comparative_images(ima_direct5_us, ima_direct, save_image, "q18_comp", "64x64
Upscaled Image")
plot_cross_sections(ima_direct5_us, ima_direct, save_image, "q18_cross", "64x64
Upscaled Image")
PSNR5 = psnr(abs(ima_direct5_us), ima_direct)
SSIM5 = ssim(abs(ima_direct5_us), ima_direct)

```

Question 1.9

```

tic()
ima9 = gridkb(spiral_data.kdata, spiral_data.ktraj, area2, N, 1, 2, 'image');
time_elapsed6 = cputime-t0
ima9 = abs(ima9).'/max(abs(ima9(:)));
% ima9 = ima9 * 1.08; % for normalisation correcting. comment to disable
plot_comparative_images(ima9, ima_direct, save_image, "q19_comp_nc", "1X Gridded
Image")
plot_cross_sections(ima9, ima_direct, save_image, "q19_cross_nc", "1X Gridded Image")
PSNR6 = psnr(abs(ima9), ima_direct)
SSIM6 = ssim(abs(ima9), ima_direct)

```

Question 1.10

```

tic()
ima10 = gridkb(spiral_data.kdata, spiral_data.ktraj, area2, N, 2, 4, 'image');
time_elapsed7 = cputime-t0
ima10 = abs(ima10).'/max(abs(ima10(:)));
figure;
imshow(ima10)
title("2X Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q110_image.png"), 'Resolution', 300)
end
ima10 = ima10(65:192, 65:192);
plot_comparative_images(ima10, ima_direct, save_image, "q110_comp", "2X Gridded
Cropped Image")
plot_cross_sections(ima10, ima_direct, save_image, "q110_cross", "2X Gridded Cropped
Image")
PSNR7 = psnr(abs(ima10), ima_direct)
SSIM7 = ssim(abs(ima10), ima_direct)

```

Question 1.11

```

tic()
ima_interp = scattered_interpolate(spiral_data.ktraj, spiral_data.kdata, 128);
time_elapsed8 = cputime-t0
plot_comparative_images(ima_interp, ima_direct, save_image, "q111_comp", '1X
Scattered Interpolation')

```

```

plot_cross_sections(ima_interp,ima_direct,save_image,"q111_cross",'1X Scattered
Interpolation')
PSNR8 = psnr(abs(ima_interp), ima_direct)
SSIM8 = ssim(abs(ima_interp), ima_direct)

```

Question 1.12

```

t0 = cputime;
time_elapsed9 = cputime-t0
ima_interp_2_full =
scattered_interpolate(spiral_data.ktraj,spiral_data.kdata,256);
ima_interp_2_crop = ima_interp_2_full(65:192, 65:192);
ima_interp_2_crop = ima_interp_2_crop / max(ima_interp_2_crop(:));

figure;
imshow(ima_interp_2_full)
title("2X Interpolated Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q112_image.png"), 'Resolution', 300)
end
plot_comparative_images(ima_interp_2_crop,ima_direct,save_image,"q112_comp",'2X
Scattered Interpolation Cropped')
plot_cross_sections(ima_interp_2_crop,ima_direct,save_image,"q112_cross",'2X
Scattered Interpolation Cropped')
PSNR9 = psnr(abs(ima_interp_2_crop), ima_direct)
SSIM9 = ssim(abs(ima_interp_2_crop), ima_direct)

```

Question 1 Analysis

```

Questions = {'1.4','1.5','1.6','1.7','1.8','1.9','1.10','1.11','1.12'};
SSIMs = num2cell([1 SSIM2 SSIM3 SSIM4 SSIM5 SSIM6 SSIM7 SSIM8 SSIM9]);
PSNRs = num2cell([inf PSNR2 PSNR3 PSNR4 PSNR5 PSNR6 PSNR7 PSNR8 PSNR9]);
times = num2cell([time_elapsed1 time_elapsed2 time_elapsed3 time_elapsed4
time_elapsed5 time_elapsed6 time_elapsed7 time_elapsed8 time_elapsed9]);
columns = {'Questions';'SSIMs';'PSNRs';'Times'};
tableq1 = [columns,[Questions;SSIMs;PSNRs;times]]

```

Question 2.1

```

P = phantom('Modified Shepp-Logan',256);
P = P/max(P(:));
figure;
imshow(P)
title("Phantom Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q21_image.png"), 'Resolution', 300)
end
subplot(1,2,1)
N = 256;
fineplot(-127:128,abs(P(end/2,:)),"Horizontal Cross Section","n_x","m[n_x,0],[-
N/2 N/2],[0 1],"on",[800 400],"Referance Image","-");

```

```

grid on
subplot(1,2,2)
fineplot(-127:128,abs(P(:,end/2)),"Vertical Cross Section","n_x","m[0,n_y],[-
N/2 N/2],[0 1],"on",[800 400],"Referance Image","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q21_cross_sections.png"),'Resolution',300)
end
proj = radon(P,0:179);
figure;
imshow(proj.',[])
xlabel("$\ell$", "Interpreter", "latex", "Position", [400 170])
ylabel("$\theta$", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)
title("Sinogram of the Phantom")

annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q21_sinogram.png"),'Resolution',300)
end

```

Question 2.2

```

inverse_proj = iradon(proj,0:179);
t0 = cputime;
inverse_proj = inverse_proj(2:257,2:257);
time_elapsed2_1 = cputime-t0
inverse_proj = inverse_proj/max(abs(inverse_proj(:)));
max(inverse_proj(:))
min(inverse_proj(:))
plot_comparetive_images(inverse_proj,P,save_image,"q22_comp","Filtered Inverse
Projection")
plot_cross_sections(inverse_proj,P,save_image,"q22_cross","Filtered Inverse
Projection","Referance Image","east")
PSNR2_1 = psnr(abs(inverse_proj),P)
SSIM2_1 = ssim(abs(inverse_proj),P)

```

Question 2.3

```

t0 = cputime;
naive_proj = iradon(proj,0:179,"None");
time_elapsed2_2 = cputime-t0
naive_proj = naive_proj(2:257,2:257);
naive_proj = naive_proj/max(naive_proj(:));
% naive_proj = naive_proj * 0.25; % for normalisation correction. comment to
disable
plot_comparetive_images(naive_proj,P,save_image,"q23_comp_nc","Naive Inverse
Projection","Referance Image",[0 1])
plot_cross_sections(naive_proj,P,save_image,"q23_cross_nc","Naive Inverse
Projection","Referance Image","east")
PSNR2_2 = psnr(abs(naive_proj),P)

```

```
SSIM2_2 = ssim(abs(naive_proj),P)
```

Question 2.4

```
figure;
kdata = fftshift(fft(ifftshift(proj,1), [], 1), 1);
imshow(log10(1+abs(kdata.')),[])
title("FFT of the Sinogram Along  $\ell$ ", "Interpreter", "latex")
xlabel(" $\rho$ ", "Interpreter", "latex", "Position", [400 170])
ylabel(" $\theta$ ", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)

annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q24_sinogram_fft.png"), 'Resolution', 300)
end
angles = [0:179]*pi/180;
radii = linspace(-0.5, 0.5, 367);
ktraj = radii.' * exp(1j*angles);
fineplot(real(ktraj), imag(ktraj), "Radial Trajectories", "k_x", "k_y", [-0.55 0.55],
[-0.55 0.55], "off", [400 400], "Trajectory", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q24_trajectory.png"), 'Resolution', 300)
end
```

Question 2.5

```
N = 180;
delta_k = 1/366;
n = 0:(367-1)/2;
d = pi/N*2*(delta_k^2)*n;
d(1) = pi/(4*N)*((delta_k)^2);
d = [flip(d) d(2:end)];
d = meshgrid(d, 1:180);
area3 = d;
figure;
fineplot(-366/2:366/2, d(1,:), "Density Compensation Filter for  $\theta =$ 
0", " $\rho$ ", "H( $\rho$ )", [-200 200], [-0.00001 0.00006], "off", [400 400], "", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q25_dcf.png"), 'Resolution', 300)
end
```

Question 2.6

```
t0 = cputime;
ima_direct = direct_summation(ktraj, kdata, d.', 256, 1);
time_elapsed2_3 = cputime-t0
ima_direct = flipud(ima_direct);
```

```

plot_comparative_images(ima_direct,P,save_image,"q26_comp","Direct Summation
Image")
plot_cross_sections(ima_direct,P,save_image,"q26_cross","Direct Summation
Image","Reference Image","east")
PSNR2_3 = psnr(abs(ima_direct),P/max(P(:)))
SSIM2_3 = ssim(abs(ima_direct),P/max(P(:)))

```

Question 2.7

```

t0 = cputime;
ima_gridding = flipplr(gridkb(kdata,ktraj,area3.',256,2,4,'image'));
time_elapsed2_4 = cputime-t0
ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
figure;
imshow(ima_gridding)
ima_gridding = ima_gridding * (1/0.7); % for normalisation correction. comment
to disable
title("Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q27_image.png"),'Resolution',300)
end
plot_comparative_images(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_c
omp_nc","Gridded Image")
plot_cross_sections(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_cross
_nc","Gridded Image","Reference Image","east")
PSNR2_4 = psnr(abs(ima_gridding(129:512-128,129:512-128)),P)
SSIM2_4 = ssim(abs(ima_gridding(129:512-128,129:512-128)),P)

```

Question 2 Analysis

```

Questions = {"2.2","2.3","2.6","2.7"};
PSNRs = num2cell([PSNR2_1 PSNR2_2 PSNR2_3 PSNR2_4]);
SSIMs = num2cell([SSIM2_1 SSIM2_2 SSIM2_3 SSIM2_4]);
times = num2cell([time_elapsed2_1 time_elapsed2_2 time_elapsed2_3
time_elapsed2_4]);
tableq2 = [columns ,[Questions;SSIMs;PSNRs;times]]

```

Question 3.1

```

radial_data = load("shepplogan_radial_data.mat");
figure;
fineplot(real(radial_data.ktraj),imag(radial_data.ktraj),"Radial
Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[400 400],"-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q31_trajectory.png"),'Resolution',300)
end

```

Question 3.2

```

d = [0:128]*(pi/300)*((0.5/128)^2)*2;

```



```

d(1) = pi/(4*300)*((0.5/128)^2);
figure;
fineplot(0:128,d,"Density Compensation Filter for \theta = 0","\rho","H(\rho)",
[-5 128],[-0.000005 0.00005],"off",[400 400],"", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q32_dcf.png"),'Resolution',300)
end
d = meshgrid(d,1:300);
area4 = d;

```

Question 3.3

```

t0 = cputime;
ima_direct = direct_summation(radial_data.ktraj,radial_data.kdata,d.',256,1);
time_elapsed3_1 = cputime-t0
ima_direct = flipud(ima_direct);
plot_comparative_images(ima_direct,P,save_image,"q33_comp","Direct Summation
Image")
plot_cross_sections(ima_direct,P,save_image,"q33_cross","Direct Summation
Image","Referance Image","east")
PSNR3_2 = psnr(abs(ima_direct),P)
SSIM3_2 = ssim(abs(ima_direct),P)

```

Question 3.4

```

t0 = cputime;
ima_gridding =
flipplr(gridkb(radial_data.kdata,radial_data.ktraj,area4.',256,2,4,'image'));
time_elapsed3_2 = cputime-t0
ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
figure;
imshow(ima_gridding)
title("Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q34_image.png"),'Resolution',300)
end
ima_gridding_cropped = ima_gridding(129:512-128,129:512-128);
ima_gridding_cropped = ima_gridding_cropped/max(ima_gridding_cropped(:));
plot_comparative_images(ima_gridding_cropped,P,save_image,"q34_comp","Gridded
Cropped Image")
plot_cross_sections(ima_gridding_cropped,P,save_image,"q34_cross","Gridded
Cropped Image","Referance Image","north")
PSNR3_2 = psnr(abs(ima_gridding_cropped),P)
SSIM3_2 = ssim(abs(ima_gridding_cropped),P)

```

Functions

Direct summation function used in Q1,2,3.

```

function ima_direct = direct_summation(ktraj,kdata,d,N,dx)
    kx = real(ktraj(:));

```

```

ky = imag(ktraj(:));
d = d(:);
M = kdata(:);

xs = ([0:N-1]-N/2)*dx;
ys = xs;

ima_direct = zeros(N,N);
for i = 1:N
    for j = 1:N
        ima_direct(i,j) = sum(d .* M .* exp(1j*2*pi*(kx*xs(j) + ky*ys(i))));
    end
end
ima_direct = abs(ima_direct);
ima_direct = ima_direct / max(abs(ima_direct(:)));
end

```

2 dimensional FFT with automatic fft shifting provided in HW1.

```

function d = fft2c(im)
    d = fftshift(fft2(ifftshift(im)));
end

```

2 dimensional inverse FFT with automatic fft shifting provided in HW1.

```

function im = ifft2c(d)
    im = fftshift(ifft2(ifftshift(d)));
end

```

Function to automatically plot overlapping cross sections of the provided image.

```

function
plot_cross_sections(im1,ref,save_image,save_str,compared_str,ref_str,location)
    arguments
        im1
        ref
        save_image
        save_str
        compared_str
        ref_str = "Referance Image"
        location = "south"
    end
    figure;
    N = size(im1);
    N = N(1);
    subplot(1,2,1)
    fineplot([1:N]-N/2,abs(im1(end/2,:)),"Horizontal Cross
Section", "n_x", "m[n_x,0]", [-N/2 N/2], [0 1], "on", [800 400], compared_str, "b-");
    fineplot([1:N]-N/2,abs(ref(end/2,:)),"Horizontal Cross
Section", "n_x", "m[n_x,0]", [-N/2 N/2], [0 1], "on", [800 400], ref_str, "r-");
    legend("Location",location)
    grid on
    subplot(1,2,2)

```

```

    fineplot([1:N]-N/2,abs(im1(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y],[-N/2 N/2],[0 1],"on",[800 400],compared_str,"b-");
    fineplot([1:N]-N/2,abs(ref(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y],[-N/2 N/2],[0 1],"on",[800 400],ref_str,"r-");
    legend("Location",location)
    grid on
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
    if save_image
        exportgraphics(gcf,strcat(save_str,".png"),'Resolution',300)
    end
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
end

```

Function to automatically plot the image along with the reference image and their error maps.

```

function
plot_comparative_images(im1,ref,save_image,save_str,compared_str,ref_str,interval
)
    arguments
        im1
        ref
        save_image
        save_str
        compared_str
        ref_str = "Reference Image"
        interval = [0 0.2]
    end
    if interval == [0 0.2]
        subplot_num = 3;
    else
        subplot_num = 4;
    end

    figure;
    subplot(1,subplot_num,1)
    imshow(im1)
    title(compared_str)
    subplot(1,subplot_num,2)
    imshow(ref)
    title(ref_str)
    subplot(1,subplot_num,3)
    imshow(abs(abs(im1) - abs(ref)),[0 0.2])
    if interval == [0 0.2]
        title("Error Map")
    else
        title("Error Map -> [0 0.2]")
    end

    if interval == [0 0.2]
        temp = 1;
    else
        subplot(1,subplot_num,4)
        imshow(abs(abs(im1) - abs(ref)),interval)
        title("Error Map -> [0 1]")
    end
end

```

```

end

set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat(save_str, ".png"), 'Resolution', 300)
end
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);

end

```

Scattered Interpolation reconstruction used in Q11, Q12.

```

function ima_interp = scattered_interpolate(ktraj, kdata, N)
    kx = real(ktraj(:));
    ky = imag(ktraj(:));
    M = kdata(:);

    kmax = 0.5;
    k_axis = linspace(-kmax, kmax, N);
    [kx_grid, ky_grid] = meshgrid(k_axis, k_axis);

    F = scatteredInterpolant(kx, ky, M, 'linear', 'none');
    k_cart = F(kx_grid, ky_grid);

    k_cart(isnan(k_cart)) = 0;
    ima_interp = ifft2c(k_cart);
    ima_interp = abs(ima_interp);
    ima_interp = ima_interp / max(ima_interp(:));
end

```

Fineplotting function used to plot more aesthetically pleasing graphs.

```

function plot_n =
fineplot(x,y,titlename,axisname,yaxisname,xlimits,ylimits,holdstate,size,legendName,style)
    arguments
        x
        y
        titlename string = ''
        axisname string = ''
        yaxisname string = ''
        xlimits (1,2) double = [min(x) max(x)]
        ylimits (1,2) double = [min(y) max(y)]
        holdstate string = 'off'
        size (1,2) double = [400 400];
        legendName string = ''
        style = '-'
    end
    % x = [x(1) x];
    % y = [y(1) y];
    %even if the limits are reverse this section corrects the order
    if ylimits(1) > ylimits(2)
        ylimits = [ylimits(2) ylimits(1)];
    end
end

```

```

end
if xlimits(1) > xlimits(2)
    xlimits = [xlimits(2) xlimits(1)];
end

%these measures are necessary for positioning arrows, labels etc.
dx = ((xlimits(2)-xlimits(1))/65);
dy = ((ylimits(2)-ylimits(1))/85);

plot_n = plot(x,y,style,'displayName',legendName);
xlim(xlimits);
ylim(ylimits);
set(get(gca,'XLabel'),'Visible','on');
set(gca,'XAxisLocation','origin','box','off');
set(gca,'YAxisLocation','origin');
set(get(gca,'XAxis'),'FontWeight','bold');
set(get(gca,'YAxis'),'FontWeight','bold');
set(get(gca,'YLabel'),'Visible','on');
set(gca,'Layer','top');
set(gcf,'position',[0, 0, size(1) , size(2)]);

%deletes the ticks that overlap with arrows
xticks('auto');
xt = xticks;
xticks(xt(2:length(xt)-1));
yticks('auto');
yt = yticks;
yticks(yt(2:length(yt)-1));

% determining the ylevel to draw the arrows on the x axis
if ((ylimits(1) * ylimits(2)) < 0)
    xal = 0;
elseif ylimits(1) < 0
    xal = ylimits(2);
else
    xal = ylimits(1);
end

%plotting the arrows
hold on
if xlimits(2) > 0
    plot([(xlimits(2)-dx) xlimits(2) (xlimits(2)-dx)],[xal+dy xal xal-
dy],'k','HandleVisibility','off') %xaxis right arrow
end
if xlimits(1) < 0
    plot([xlimits(1)+dx xlimits(1) xlimits(1)+dx],[xal+dy xal xal-
dy],'k','HandleVisibility','off') %xaxis left arrow
end
plot([0 0],[ylimits(2) ylimits(1)],'k','HandleVisibility','off')% y axis
if ylimits(2) > 0
    plot([-dx/2 0 dx/2],[ylimits(2)-dy ylimits(2) ylimits(2)-
dy],'k','HandleVisibility','off') %yaxis top arrow
end
if ylimits(1) < 0

```

```

        plot([-dx/2 0 dx/2],[ylimits(1)+dy ylimits(1)
ylimits(1)+dy],'k','HandleVisibility','off') %yaxis bottom arrow
    end

    %repositioning title & label locations
    label_h1 = xlabel(xaxisname);
    label_h1.Position(1) = xlims(2)+dx; % change horizontal position of xlabel.
    label_h1.Position(2) = dy; % change vertical position of xlabel.c5
    label_h2 = ylabel(yaxisname,rotation=0);
    label_h2.Position(1) = dx; % change horizontal position of ylabel.
    label_h2.Position(2) = ylimits(2)+dy; % change vertical position of ylabel.
    title(titlename);

    %holds or doesnt hold
    if strcmp(holdstate,'off')
        hold off
    else
        hold on
    end
end
end

```

```
save_image = 0
```

Question 1.1

```
spiral_data = load("spiral.mat");
figure;
fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"K-Space
Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[600 900],"","-");
grid on

set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q11_trajectory.png"), 'Resolution', 300)
end
```

Question 1.2

```
area = voronoidens(spiral_data.ktraj);
total_nans = sum(isnan(area), 'all')
subplot(1,2,1)
fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample", "Area", [-500
length(area(:))], [-0.02 0.1], "off", [800 400], "", "-");
grid on
subplot(1,2,2)
fineplot(1:length(area(:)),area(:),"Area of Each Cell","#sample", "Area", [-500
length(area(:))], [-0.00004 0.0002], "off", [800 400], "", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q12_area.png"), 'Resolution', 300)
end
```

Question 1.3

```
figure;
circle = 0.51*exp(1*j*linspace(0,2*pi,2049));
circle = circle(1:end-1);
limited_ktraj=[spiral_data.ktraj circle.'];
fineplot(real(spiral_data.ktraj),imag(spiral_data.ktraj),"Boundary and K-
Space Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"on",[400
800], "Trajectory", "-");
plot(circle, "LineWidth", 5, 'displayName', "Boundary")
legend()
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q13_trajectory.png"), 'Resolution', 300)
end
figure;
area2 = voronoidens(limited_ktraj);
area2 = area2(:,1:6);
subplot(1,3,1:2)
fineplot(1:length(area2(:)),area2(:),"Area of Each Cell","#sample", "Area", [-500
length(area2(:))], [-0.00004 0.0002], "off", [800 400], "", "-");
```



```

grid on
subplot(1,3,3)
fineplot(1:length(area2(:,1)),area2(:,1),"Area of Single
Trajectory","#sample","Area",[-500 length(area2(:,1))],[-0.00004 0.0002],"off",
[800 400],"","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q13_area.png"), 'Resolution', 300)
end

```

Question 1.4

```

N=128;
t0 = cputime;
ima_direct = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,128,1);
time_elapsed1 = cputime-t0
figure;
imshow(ima_direct)
title("Reference Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q14_image.png"), 'Resolution', 300)
end
figure;
subplot(1,2,1)
fineplot([1:N]-N/2,abs(ima_direct(end/2,:)),"Horizontal Cross
Section","n_x","m[n_x,0]",[-N/2 N/2],[0 1],"off",[800 400],"","-");
grid on
subplot(1,2,2)
fineplot([1:N]-N/2,abs(ima_direct(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y]",[-N/2 N/2],[0 1],"off",[800 400],"","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q14_cross.png"), 'Resolution', 300)
end

```

Question 1.5

```

N=128;
t0 = cputime; tic()
ima_direct2 =
direct_summation(spiral_data.ktraj,spiral_data.kdata,ones(size(spiral_data.kdata)
),128,1);
time_elapsed2 = cputime-t0
plot_comparative_images(ima_direct2,ima_direct,save_image,"q15_comp","No Density
Compensation","Reference Image",[0 1])
plot_cross_sections(ima_direct2,ima_direct,save_image,"q15_cross","No Density
Compensation")
PSNR2 = psnr(abs(ima_direct2),ima_direct)
SSIM2 = ssim(abs(ima_direct2),ima_direct)

```

Question 1.6

```
N3 = 256;
t0 = cputime;
ima_direct3 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,256,1);
time_elapsed3 = cputime-t0
cropped = ima_direct3(N3/4+1:3*N3/4,N3/4+1:3*N3/4);
figure;
imshow(ima_direct3)
title("256 x 256 Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q16_image.png"), 'Resolution', 300)
end
plot_comparative_images(cropped,ima_direct,save_image,"q16_comp","256x256 Direct
Summed Cropped Image")
plot_cross_sections(cropped,ima_direct,save_image,"q16_cross","256x256 Direct
Summed Cropped Image")
PSNR3 = psnr(abs(cropped),ima_direct)
SSIM3 = ssim(abs(cropped),ima_direct)
```

Question 1.7

```
N4 = 256;
t0 = cputime;
ima_direct4 =
direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,256,0.5);
time_elapsed4 = cputime-t0
k_ref = fft2c(ima_direct);
k_ref_padded = zeros([N4 N4]);
k_ref_padded(N4/4+1:(3*N4/4),N4/4+1:(3*N4/4)) = k_ref;
ima_direct_us = ifft2c(k_ref_padded);
ima_direct_us = ima_direct_us/max(ima_direct_us(:));
plot_comparative_images(abs(ima_direct4),abs(ima_direct_us),save_image,"q17_comp"
,"256x256 Direct Summed Image","Upscaled Reference Image")
plot_cross_sections(ima_direct4,ima_direct_us,save_image,"q17_cross","256x256
Direct Summed Image","Upscaled Reference Image")
PSNR4 = psnr(abs(ima_direct4),abs(ima_direct_us))
SSIM4 = ssim(abs(ima_direct4),abs(ima_direct_us))
```

Question 1.8

```
t0 = cputime;
ima_direct5 = direct_summation(spiral_data.ktraj,spiral_data.kdata,area2,64,2);
time_elapsed5 = cputime-t0
k_data_5 = fft2c(ima_direct5);
k_data_5_padded = zeros([N,N]);
k_data_5_padded(N/4+1:3*N/4,N/4+1:3*N/4) = k_data_5;
ima_direct5_us = ifft2c(k_data_5_padded);
ima_direct5_us = abs(ima_direct5_us)/max(abs(ima_direct5_us(:)));
% ima_direct5_us = ima_direct5_us * 1.2; %normalisation correction? comment to
disable
figure;
imshow(ima_direct5)
```

```

title("64x64 Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q18_image.png"), 'Resolution', 300)
end
plot_comparative_images(ima_direct5_us, ima_direct, save_image, "q18_comp", "64x64
Upscaled Image")
plot_cross_sections(ima_direct5_us, ima_direct, save_image, "q18_cross", "64x64
Upscaled Image")
PSNR5 = psnr(abs(ima_direct5_us), ima_direct)
SSIM5 = ssim(abs(ima_direct5_us), ima_direct)

```

Question 1.9

```

tic()
ima9 = gridkb(spiral_data.kdata, spiral_data.ktraj, area2, N, 1, 2, 'image');
time_elapsed6 = cputime-t0
ima9 = abs(ima9).'/max(abs(ima9(:)));
% ima9 = ima9 * 1.08; % for normalisation correcting. comment to disable
plot_comparative_images(ima9, ima_direct, save_image, "q19_comp_nc", "1X Gridded
Image")
plot_cross_sections(ima9, ima_direct, save_image, "q19_cross_nc", "1X Gridded Image")
PSNR6 = psnr(abs(ima9), ima_direct)
SSIM6 = ssim(abs(ima9), ima_direct)

```

Question 1.10

```

tic()
ima10 = gridkb(spiral_data.kdata, spiral_data.ktraj, area2, N, 2, 4, 'image');
time_elapsed7 = cputime-t0
ima10 = abs(ima10).'/max(abs(ima10(:)));
figure;
imshow(ima10)
title("2X Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q110_image.png"), 'Resolution', 300)
end
ima10 = ima10(65:192, 65:192);
plot_comparative_images(ima10, ima_direct, save_image, "q110_comp", "2X Gridded
Cropped Image")
plot_cross_sections(ima10, ima_direct, save_image, "q110_cross", "2X Gridded Cropped
Image")
PSNR7 = psnr(abs(ima10), ima_direct)
SSIM7 = ssim(abs(ima10), ima_direct)

```

Question 1.11

```

tic()
ima_interp = scattered_interpolate(spiral_data.ktraj, spiral_data.kdata, 128);
time_elapsed8 = cputime-t0
plot_comparative_images(ima_interp, ima_direct, save_image, "q111_comp", '1X
Scattered Interpolation')

```

```

plot_cross_sections(ima_interp,ima_direct,save_image,"q111_cross",'1X Scattered
Interpolation')
PSNR8 = psnr(abs(ima_interp), ima_direct)
SSIM8 = ssim(abs(ima_interp), ima_direct)

```

Question 1.12

```

t0 = cputime;
time_elapsed9 = cputime-t0
ima_interp_2_full =
scattered_interpolate(spiral_data.ktraj,spiral_data.kdata,256);
ima_interp_2_crop = ima_interp_2_full(65:192, 65:192);
ima_interp_2_crop = ima_interp_2_crop / max(ima_interp_2_crop(:));

figure;
imshow(ima_interp_2_full)
title("2X Interpolated Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q112_image.png"), 'Resolution', 300)
end
plot_comparative_images(ima_interp_2_crop,ima_direct,save_image,"q112_comp",'2X
Scattered Interpolation Cropped')
plot_cross_sections(ima_interp_2_crop,ima_direct,save_image,"q112_cross",'2X
Scattered Interpolation Cropped')
PSNR9 = psnr(abs(ima_interp_2_crop), ima_direct)
SSIM9 = ssim(abs(ima_interp_2_crop), ima_direct)

```

Question 1 Analysis

```

Questions = {'1.4','1.5','1.6','1.7','1.8','1.9','1.10','1.11','1.12'};
SSIMs = num2cell([1 SSIM2 SSIM3 SSIM4 SSIM5 SSIM6 SSIM7 SSIM8 SSIM9]);
PSNRs = num2cell([inf PSNR2 PSNR3 PSNR4 PSNR5 PSNR6 PSNR7 PSNR8 PSNR9]);
times = num2cell([time_elapsed1 time_elapsed2 time_elapsed3 time_elapsed4
time_elapsed5 time_elapsed6 time_elapsed7 time_elapsed8 time_elapsed9]);
columns = {'Questions';'SSIMs';'PSNRs';'Times'};
tableq1 = [columns,[Questions;SSIMs;PSNRs;times]]

```

Question 2.1

```

P = phantom('Modified Shepp-Logan',256);
P = P/max(P(:));
figure;
imshow(P)
title("Phantom Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q21_image.png"), 'Resolution', 300)
end
subplot(1,2,1)
N = 256;
fineplot(-127:128,abs(P(end/2,:)),"Horizontal Cross Section","n_x","m[n_x,0],[-
N/2 N/2],[0 1],"on",[800 400],"Referance Image","-");

```

```

grid on
subplot(1,2,2)
fineplot(-127:128,abs(P(:,end/2)),"Vertical Cross Section","n_x","m[0,n_y],[-
N/2 N/2],[0 1],"on",[800 400],"Referance Image","-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q21_cross_sections.png"),'Resolution',300)
end
proj = radon(P,0:179);
figure;
imshow(proj.',[])
xlabel("$\ell$", "Interpreter", "latex", "Position", [400 170])
ylabel("$\theta$", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)
title("Sinogram of the Phantom")

annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q21_sinogram.png"),'Resolution',300)
end

```

Question 2.2

```

inverse_proj = iradon(proj,0:179);
t0 = cputime;
inverse_proj = inverse_proj(2:257,2:257);
time_elapsed2_1 = cputime-t0
inverse_proj = inverse_proj/max(abs(inverse_proj(:)));
max(inverse_proj(:))
min(inverse_proj(:))
plot_comparetive_images(inverse_proj,P,save_image,"q22_comp","Filtered Inverse
Projection")
plot_cross_sections(inverse_proj,P,save_image,"q22_cross","Filtered Inverse
Projection","Referance Image","east")
PSNR2_1 = psnr(abs(inverse_proj),P)
SSIM2_1 = ssim(abs(inverse_proj),P)

```

Question 2.3

```

t0 = cputime;
naive_proj = iradon(proj,0:179,"None");
time_elapsed2_2 = cputime-t0
naive_proj = naive_proj(2:257,2:257);
naive_proj = naive_proj/max(naive_proj(:));
% naive_proj = naive_proj * 0.25; % for normalisation correction. comment to
disable
plot_comparetive_images(naive_proj,P,save_image,"q23_comp_nc","Naive Inverse
Projection","Referance Image",[0 1])
plot_cross_sections(naive_proj,P,save_image,"q23_cross_nc","Naive Inverse
Projection","Referance Image","east")
PSNR2_2 = psnr(abs(naive_proj),P)

```

```
SSIM2_2 = ssim(abs(naive_proj),P)
```

Question 2.4

```
figure;
kdata = fftshift(fft(ifftshift(proj,1), [], 1), 1);
imshow(log10(1+abs(kdata.')),[])
title("FFT of the Sinogram Along  $\ell$ ", "Interpreter", "latex")
xlabel(" $\rho$ ", "Interpreter", "latex", "Position", [400 170])
ylabel(" $\theta$ ", "Interpreter", "latex", "Position", [10 0.5], "Rotation", 0)

annotation("arrow", [0.1634 0.1634], [0.2636 0.9335]) % left
annotation("arrow", [0.1644 0.8836], [0.2636 0.2636]) % bottom
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q24_sinogram_fft.png"), 'Resolution', 300)
end
angles = [0:179]*pi/180;
radii = linspace(-0.5, 0.5, 367);
ktraj = radii.' * exp(1j*angles);
fineplot(real(ktraj), imag(ktraj), "Radial Trajectories", "k_x", "k_y", [-0.55 0.55],
[-0.55 0.55], "off", [400 400], "Trajectory", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q24_trajectory.png"), 'Resolution', 300)
end
```

Question 2.5

```
N = 180;
delta_k = 1/366;
n = 0:(367-1)/2;
d = pi/N*2*(delta_k^2)*n;
d(1) = pi/(4*N)*((delta_k)^2);
d = [flip(d) d(2:end)];
d = meshgrid(d, 1:180);
area3 = d;
figure;
fineplot(-366/2:366/2, d(1,:), "Density Compensation Filter for  $\theta =$ 
0", " $\rho$ ", "H( $\rho$ )", [-200 200], [-0.00001 0.00006], "off", [400 400], "", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf, strcat("q25_dcf.png"), 'Resolution', 300)
end
```

Question 2.6

```
t0 = cputime;
ima_direct = direct_summation(ktraj, kdata, d.', 256, 1);
time_elapsed2_3 = cputime-t0
ima_direct = flipud(ima_direct);
```

```

plot_comparative_images(ima_direct,P,save_image,"q26_comp","Direct Summation
Image")
plot_cross_sections(ima_direct,P,save_image,"q26_cross","Direct Summation
Image","Reference Image","east")
PSNR2_3 = psnr(abs(ima_direct),P/max(P(:)))
SSIM2_3 = ssim(abs(ima_direct),P/max(P(:)))

```

Question 2.7

```

t0 = cputime;
ima_gridding = flipplr(gridkb(kdata,ktraj,area3.',256,2,4,'image'));
time_elapsed2_4 = cputime-t0
ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
figure;
imshow(ima_gridding)
ima_gridding = ima_gridding * (1/0.7); % for normalisation correction. comment
to disable
title("Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q27_image.png"),'Resolution',300)
end
plot_comparative_images(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_c
omp_nc","Gridded Image")
plot_cross_sections(ima_gridding(129:512-128,129:512-128),P,save_image,"q27_cross
_nc","Gridded Image","Reference Image","east")
PSNR2_4 = psnr(abs(ima_gridding(129:512-128,129:512-128)),P)
SSIM2_4 = ssim(abs(ima_gridding(129:512-128,129:512-128)),P)

```

Question 2 Analysis

```

Questions = {"2.2","2.3","2.6","2.7"};
PSNRs = num2cell([PSNR2_1 PSNR2_2 PSNR2_3 PSNR2_4]);
SSIMs = num2cell([SSIM2_1 SSIM2_2 SSIM2_3 SSIM2_4]);
times = num2cell([time_elapsed2_1 time_elapsed2_2 time_elapsed2_3
time_elapsed2_4]);
tableq2 = [columns ,[Questions;SSIMs;PSNRs;times]]

```

Question 3.1

```

radial_data = load("shepplogan_radial_data.mat");
figure;
fineplot(real(radial_data.ktraj),imag(radial_data.ktraj),"Radial
Trajectories","k_x","k_y",[-0.55 0.55],[-0.55 0.55],"off",[400 400],"-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q31_trajectory.png"),'Resolution',300)
end

```

Question 3.2

```

d = [0:128]*(pi/300)*((0.5/128)^2)*2;

```



```

d(1) = pi/(4*300)*((0.5/128)^2);
figure;
fineplot(0:128,d,"Density Compensation Filter for \theta = 0","\rho","H(\rho)",
[-5 128],[-0.000005 0.00005],"off",[400 400],"", "-");
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q32_dcf.png"),'Resolution',300)
end
d = meshgrid(d,1:300);
area4 = d;

```

Question 3.3

```

t0 = cputime;
ima_direct = direct_summation(radial_data.ktraj,radial_data.kdata,d.',256,1);
time_elapsed3_1 = cputime-t0
ima_direct = flipud(ima_direct);
plot_comparative_images(ima_direct,P,save_image,"q33_comp","Direct Summation
Image")
plot_cross_sections(ima_direct,P,save_image,"q33_cross","Direct Summation
Image","Referance Image","east")
PSNR3_2 = psnr(abs(ima_direct),P)
SSIM3_2 = ssim(abs(ima_direct),P)

```

Question 3.4

```

t0 = cputime;
ima_gridding =
flipplr(gridkb(radial_data.kdata,radial_data.ktraj,area4.',256,2,4,'image'));
time_elapsed3_2 = cputime-t0
ima_gridding = abs(ima_gridding).'/max(abs(ima_gridding(:)));
figure;
imshow(ima_gridding)
title("Gridded Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
if save_image
    exportgraphics(gcf,strcat("q34_image.png"),'Resolution',300)
end
ima_gridding_cropped = ima_gridding(129:512-128,129:512-128);
ima_gridding_cropped = ima_gridding_cropped/max(ima_gridding_cropped(:));
plot_comparative_images(ima_gridding_cropped,P,save_image,"q34_comp","Gridded
Cropped Image")
plot_cross_sections(ima_gridding_cropped,P,save_image,"q34_cross","Gridded
Cropped Image","Referance Image","north")
PSNR3_2 = psnr(abs(ima_gridding_cropped),P)
SSIM3_2 = ssim(abs(ima_gridding_cropped),P)

```

Functions

Direct summation function used in Q1,2,3.

```

function ima_direct = direct_summation(ktraj,kdata,d,N,dx)
    kx = real(ktraj(:));

```

```

ky = imag(ktraj(:));
d = d(:);
M = kdata(:);

xs = ([0:N-1]-N/2)*dx;
ys = xs;

ima_direct = zeros(N,N);
for i = 1:N
    for j = 1:N
        ima_direct(i,j) = sum(d .* M .* exp(1j*2*pi*(kx*xs(j) + ky*ys(i))));
    end
end
ima_direct = abs(ima_direct);
ima_direct = ima_direct / max(abs(ima_direct(:)));
end

```

2 dimensional FFT with automatic fft shifting provided in HW1.

```

function d = fft2c(im)
    d = fftshift(fft2(ifftshift(im)));
end

```

2 dimensional inverse FFT with automatic fft shifting provided in HW1.

```

function im = ifft2c(d)
    im = fftshift(ifft2(ifftshift(d)));
end

```

Function to automatically plot overlapping cross sections of the provided image.

```

function
plot_cross_sections(im1,ref,save_image,save_str,compared_str,ref_str,location)
    arguments
        im1
        ref
        save_image
        save_str
        compared_str
        ref_str = "Referance Image"
        location = "south"
    end
    figure;
    N = size(im1);
    N = N(1);
    subplot(1,2,1)
    fineplot([1:N]-N/2,abs(im1(end/2,:)),"Horizontal Cross
Section", "n_x", "m[n_x,0]", [-N/2 N/2], [0 1], "on", [800 400], compared_str, "b-");
    fineplot([1:N]-N/2,abs(ref(end/2,:)),"Horizontal Cross
Section", "n_x", "m[n_x,0]", [-N/2 N/2], [0 1], "on", [800 400], ref_str, "r-");
    legend("Location",location)
    grid on
    subplot(1,2,2)

```

```

    fineplot([1:N]-N/2,abs(im1(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y],[-N/2 N/2],[0 1],"on",[800 400],compared_str,"b-");
    fineplot([1:N]-N/2,abs(ref(:,end/2)),"Vertical Cross
Section","n_x","m[0,n_y],[-N/2 N/2],[0 1],"on",[800 400],ref_str,"r-");
    legend("Location",location)
    grid on
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
    if save_image
        exportgraphics(gcf,strcat(save_str,".png"),'Resolution',300)
    end
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.6, 0.3]);
end

```

Function to automatically plot the image along with the reference image and their error maps.

```

function
plot_comparative_images(im1,ref,save_image,save_str,compared_str,ref_str,interval
)
    arguments
        im1
        ref
        save_image
        save_str
        compared_str
        ref_str = "Reference Image"
        interval = [0 0.2]
    end
    if interval == [0 0.2]
        subplot_num = 3;
    else
        subplot_num = 4;
    end

    figure;
    subplot(1,subplot_num,1)
    imshow(im1)
    title(compared_str)
    subplot(1,subplot_num,2)
    imshow(ref)
    title(ref_str)
    subplot(1,subplot_num,3)
    imshow(abs(abs(im1) - abs(ref)),[0 0.2])
    if interval == [0 0.2]
        title("Error Map")
    else
        title("Error Map -> [0 0.2]")
    end

    if interval == [0 0.2]
        temp = 1;
    else
        subplot(1,subplot_num,4)
        imshow(abs(abs(im1) - abs(ref)),interval)
        title("Error Map -> [0 1]")
    end
end

```

```

end

set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat(save_str, ".png"), 'Resolution', 300)
end
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);

end

```

Scattered Interpolation reconstruction used in Q11, Q12.

```

function ima_interp = scattered_interpolate(ktraj, kdata, N)
    kx = real(ktraj(:));
    ky = imag(ktraj(:));
    M = kdata(:);

    kmax = 0.5;
    k_axis = linspace(-kmax, kmax, N);
    [kx_grid, ky_grid] = meshgrid(k_axis, k_axis);

    F = scatteredInterpolant(kx, ky, M, 'linear', 'none');
    k_cart = F(kx_grid, ky_grid);

    k_cart(isnan(k_cart)) = 0;
    ima_interp = ifft2c(k_cart);
    ima_interp = abs(ima_interp);
    ima_interp = ima_interp / max(ima_interp(:));
end

```

Fineplotting function used to plot more aesthetically pleasing graphs.

```

function plot_n =
fineplot(x,y,titlename,axisname,yaxisname,xlimits,ylimits,holdstate,size,legendName,style)
    arguments
        x
        y
        titlename string = ''
        axisname string = ''
        yaxisname string = ''
        xlimits (1,2) double = [min(x) max(x)]
        ylimits (1,2) double = [min(y) max(y)]
        holdstate string = 'off'
        size (1,2) double = [400 400];
        legendName string = ''
        style = '-'
    end
    % x = [x(1) x];
    % y = [y(1) y];
    %even if the limits are reverse this section corrects the order
    if ylimits(1) > ylimits(2)
        ylimits = [ylimits(2) ylimits(1)];
    end
end

```

```

end
if xlimits(1) > xlimits(2)
    xlimits = [xlimits(2) xlimits(1)];
end

%these measures are necessary for positioning arrows, labels etc.
dx = ((xlimits(2)-xlimits(1))/65);
dy = ((ylimits(2)-ylimits(1))/85);

plot_n = plot(x,y,style,'displayName',legendName);
xlim(xlimits);
ylim(ylimits);
set(get(gca,'XLabel'),'Visible','on');
set(gca,'XAxisLocation','origin','box','off');
set(gca,'YAxisLocation','origin');
set(get(gca,'XAxis'),'FontWeight','bold');
set(get(gca,'YAxis'),'FontWeight','bold');
set(get(gca,'YLabel'),'Visible','on');
set(gca,'Layer','top');
set(gcf,'position',[0, 0, size(1) , size(2)]);

%deletes the ticks that overlap with arrows
xticks('auto');
xt = xticks;
xticks(xt(2:length(xt)-1));
yticks('auto');
yt = yticks;
yticks(yt(2:length(yt)-1));

% determining the ylevel to draw the arrows on the x axis
if ((ylimits(1) * ylimits(2)) < 0)
    xal = 0;
elseif ylimits(1) < 0
    xal = ylimits(2);
else
    xal = ylimits(1);
end

%plotting the arrows
hold on
if xlimits(2) > 0
    plot([(xlimits(2)-dx) xlimits(2) (xlimits(2)-dx)],[xal+dy xal xal-
dy],'k','HandleVisibility','off') %xaxis right arrow
end
if xlimits(1) < 0
    plot([xlimits(1)+dx xlimits(1) xlimits(1)+dx],[xal+dy xal xal-
dy],'k','HandleVisibility','off') %xaxis left arrow
end
plot([0 0],[ylimits(2) ylimits(1)],'k','HandleVisibility','off')% y axis
if ylimits(2) > 0
    plot([-dx/2 0 dx/2],[ylimits(2)-dy ylimits(2) ylimits(2)-
dy],'k','HandleVisibility','off') %yaxis top arrow
end
if ylimits(1) < 0

```

```

        plot([-dx/2 0 dx/2],[ylimits(1)+dy ylimits(1)
ylimits(1)+dy],'k','HandleVisibility','off') %yaxis bottom arrow
    end

    %repositioning title & label locations
    label_h1 = xlabel(xaxisname);
    label_h1.Position(1) = xlims(2)+dx; % change horizontal position of xlabel.
    label_h1.Position(2) = dy; % change vertical position of xlabel.c5
    label_h2 = ylabel(yaxisname,rotation=0);
    label_h2.Position(1) = dx; % change horizontal position of ylabel.
    label_h2.Position(2) = ylimits(2)+dy; % change vertical position of ylabel.
    title(titlename);

    %holds or doesnt hold
    if strcmp(holdstate,'off')
        hold off
    else
        hold on
    end
end
end

```